

Which coin is the real one, while it's still climbing.

What the user sees, what runs behind the scenes, how each part gets built, and how long it honestly takes.

PART ONE	PART TWO	PART THREE	PART FOUR
The product Every screen, and what a trader does with it.	The machinery Detection, grouping, matching, and how it learns. None of it visible.	How we build it The stack, the structure, the order, the timeline.	How it comes together Where the line sits, and one story end to end.

Contents

PART ONE • THE PRODUCT

01	What Insidor is	The problem with every existing tool, and the window we can actually win.
02	The feed	The home screen. Seven things per row, and nothing else.
03	The story page	The main attraction. Every coin, labelled honestly, plus the discussion.
04	Create a coin	Minting off a story, without ever holding a key.
05	Trading, watchlist, search, trending	The supporting screens.

PART TWO • THE MACHINERY

06	How we find viral posts	One post in a thousand, and the moment a post escapes its own audience.
07	How posts become stories	Memes get copied. News gets consumed. That difference is the detector.
08	How stories become coins	One meme, 306 tokens. Picking the real one is the buy path.
09	How the system learns	Two tables that cannot be added later.
10	What we store, and what it costs	The database in plain English, and the scores we never show.

PART THREE • HOW WE BUILD IT

11	The stack, and why	The bug that settles the language question.
12	Where the code lives	The tree alone should tell you where to look.
13	How each part gets built	Eleven steps, what each unblocks, and how you know it’s done.
14	The timeline, honestly	Four months — and the week after which a slip stops costing the asset.

PART FOUR • HOW IT COMES TOGETHER

15	Where the line sits	Facts go on screen. Our reasoning never does.
16	One story, end to end	The ferry captain, and the two windows — one we lose, one we win.

WHAT CHANGED SINCE THE LAST VERSION

We measured the real timing. A coin is minted about **two to four minutes** after the post it came from, so nobody beats the bots to it and the old claim — that we surface a story before any coin exists — is not true for the coins that make money. But a coin takes about **six days** to peak, and one meme spawns hundreds of copies. So the claim that survives is stronger and harder to copy: **we tell you which of those coins is the real one, while it is still climbing.**

PART ONE

The product

What a trader sees and does. Every screen in this part shows facts about the world and nothing about how we decided.

-
- 01 What Insider is
 - 02 The feed
 - 03 The story page
 - 04 Create a coin
 - 05 Trading, watchlist, search, trending

01. What Insidor is

Memecoins are made out of things going viral. Every tool a trader uses today starts at the money — it can only show a coin once the coin exists and buying has begun. Insidor starts at the internet instead: we watch the posts, and when coins appear off one of them we tell you which one is the real one, while there is still a run left to catch.

What we are no longer claiming

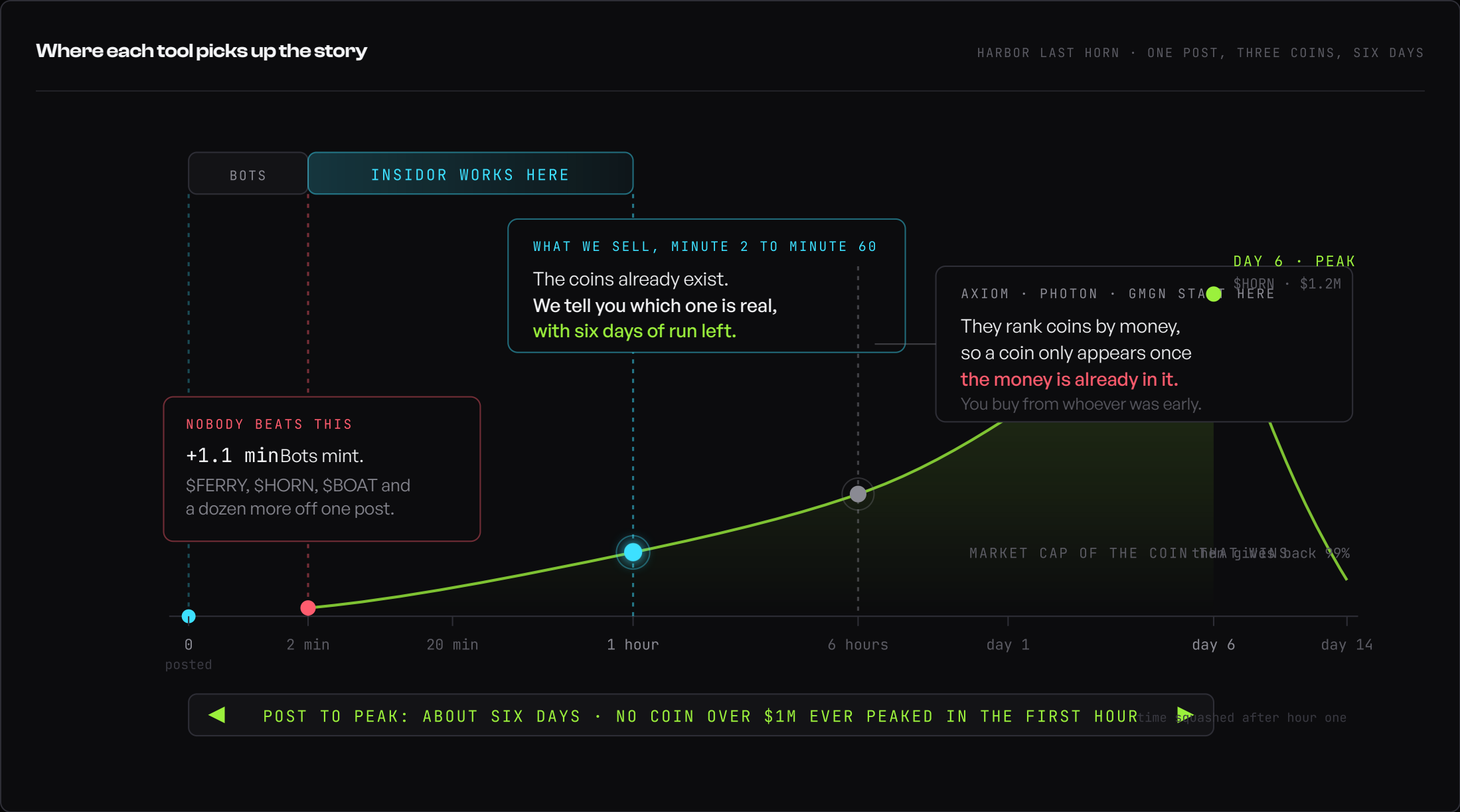
An earlier version of this deck said we would show you the story *before the coin exists*. We measured it, and that is not true for any coin worth owning. Post IDs on X carry their own timestamp, so the gap between a post and the coin minted from it can be computed exactly. The median is 3.8 minutes. For coins that went on to clear a million dollars it is **1.1 minutes**, and 88% of them were minted inside the first hour. The slower the mint, the worse the coin did. Chill Guy's famous thirteen-month gap turned out to be an old tweet dug up and re-minted years later, and every case like it topped out between \$159K and \$1.8M.

So the first two minutes belong to bots and always will. There is no product in racing them, and pretending otherwise would have sent us building the wrong thing for four months.

What is actually still open

Two measurements say the opportunity moved rather than disappeared. First, a story does not produce one coin — it produces a crowd of them. Searching pump.fun for "chill guy" returns **306 separate tokens**. One of those made people money and 305 did not, and nothing on the coin itself tells you which. Only about 5% of coins above \$50K even declare the post they came from, so you cannot look it up.

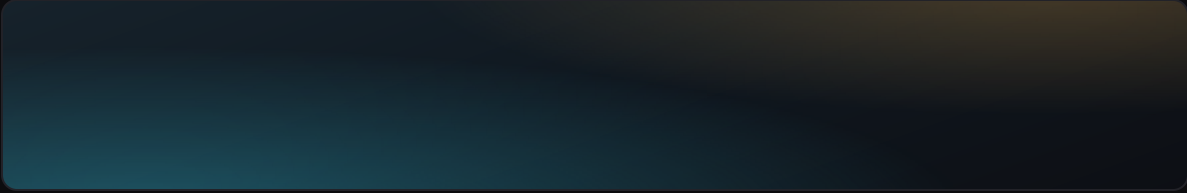
Second, the run is long. From the original post to the coin's peak is about **six days**, and of every coin that cleared a million dollars, **none** peaked inside the first hour. The mint is over in minutes; the move is not. That is the whole shape of the business: you cannot be first to the coin, but you can be early to the right one, and "early" now means hours rather than seconds.



One story, one clock. The red line is the part we lost and were never going to win. The cyan band is the hour we can own. The lime bar is why owning it is worth anything: the mint is a two-minute event, the move takes a week.

What that looks like on a screen

The user opens Insidor and sees a story explained in one ordinary sentence, the post it came from, and the coins that have already been minted off it — with the one thing no other screen can tell them: which was first, which is biggest, and which has no traceable link back to the post at all.



A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

@nordkyst_ferge on X · 2.4M views · 4h old
also a TikTok clip by @harbourclips · 890K views

COIN	MINTED	MARKET CAP		
\$FERRY	09:02 · 3h ago	\$799.6K	FIRST	Buy
\$HORN	09:06 · 3h ago	\$1.2M	BIGGEST	Buy
\$BOAT	09:11 · 3h ago	\$41.2K	UNVERIFIED	NO LINK TO THE POST

Every number here is true whether or not Insidor exists: mint times from the chain, market caps from the market, views from the platforms.

Why it is defensible

Being right about which coin is real is not a feature anyone can copy in a fortnight, because it rests on two things that only get bigger while the product runs.

COMPOUND #1 · THE COMMUNITY

Their thesis history lives here

Traders argue on each story about which of the coins is the real one, in public, under their own name. Over months that becomes a track record you cannot fake and cannot take with you.

People do not walk away from a reputation they spent a year building. The writing is the lock-in, and we are not the ones who have to produce it.

COMPOUND #2 · THE RECORD

Which posts became coins that made money

Every story we surface we keep: what the post looked like as it climbed, which coins appeared off it, and how each one ended. Nobody sells this data. You cannot go back and ask what a post looked like twenty minutes ago — if it was not recorded live, it is gone forever.

A competitor can copy the screen in a fortnight. They cannot copy a year of yesterdays.

Who this is for

THE TRADER

Wants the right coin, not the fast one

Already reads charts, already has Axiom open. What they cannot get anywhere is a reason to pick one of the fourteen coins named after the same video an hour after it was posted.

THE CREATOR

Wants to mint off a meme

Finding the funny thing early is their whole job, and today it means scrolling all day and hoping. Here the stories arrive already sorted, with what has been minted off each.

THE NEWCOMER

Saw it on TikTok, wants in

No wallet, no jargon, no idea what a bonding curve is. They get the story in one sentence, one coin recommended over the copies, and a button that works.

Three people, one screen. The trader reads it for the pick, the creator for the raw material, the newcomer for the sentence at the top. Nothing on the page is written for an insider only.

02. The feed

The home screen. One list of the stories going viral right now, best one at the top, each explained in two lines of plain English so you never have to watch the video to know what it is. Seven things per row. Nothing about how we picked them.

Every other tool a trader opens — Axiom, Photon, GMGN — is a list of coins. Insidor's home screen is a list of **stories**. Some of them have a coin, some have three, some have none. The list is what the coins are *about*, which is the thing nobody else shows you and the only thing that determines whether a coin keeps going.

The promise we had to correct

An earlier version of this deck said the feed shows you the story *before the coin exists*. We measured it, and that is not true for the coins that make money. Every X post carries its own timestamp inside its ID, so the gap between a post going up and a coin being minted from it can be computed exactly rather than guessed. The median gap is 3.8 minutes. For coins that went on to clear \$10M it is 1.1 minutes, and 88% of them were minted inside the first hour. Automated minters are watching the same firehose and they will always be first. That race is lost and it was never worth entering.

x

WHAT WE USED TO SAY

"We show you the story before the coin exists."

Measured and false where it matters. Median post-to-mint is **3.8 minutes**; for coins that reached \$10M it is **1.1 minutes**. Bots mint within a few minutes of a post going up. The one famous slow case — a meme minted thirteen months later — turned out to be the shape of the **losing** regime: every coin we found minted more than thirty days after its post was an old tweet dug up and re-used, and none of them got far.

✓

WHAT WE CAN ACTUALLY SAY

"Of the things minted in the last hour, this is the real story — and it is still climbing."

A post takes about **six days** to reach its own peak, and **zero percent** of coins that cleared \$1M peaked inside their first hour. So the mint is the starting gun, not the finish. Minutes one to ten belong to machines. Hour one to day six is where a human with a clear read on the story can still be early, and that is the window the feed is built for.



The two lines are the product

The most important text in the whole app is the sentence under each title. "A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back." Read that and you already know whether it is worth a coin — you can feel whether it is funny, whether it has a name in it, whether people will repeat it. So it is set larger and brighter than the title above it. It is not a subtitle. It is the row.

Everything else on the row is a fact about the world that would be true whether or not Insidor existed: how many people have seen it, whether the line is still going up, how old it is, and what you can do about it. Seven things, no eighth. The order of the list is where all our work goes, and the order is the only place it shows.

INSIDOR

Feed

Search

Watchlist

You

Q

Search stories, coins, people

⌵K

Connect wallet

Happening now

Tuesday 09:13

LIVE

Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

2.4M

VIEWS

4h
OLD

3 coins →

Paid to Move to Norway

A Norwegian town is paying people €25,000 to move there.

1.1M

VIEWS

1h
OLD

Create coin

Plastic Plant Grandma

A grandmother has been watering a plastic plant for eleven years and nobody told her.

850K

VIEWS

2h
OLD

Buy \$PLANT

Grandmaster vs Pigeon

A chess grandmaster lost to a pigeon and wrote an essay about it.

1.1M

VIEWS

6h
OLD

Buy \$PIGEON

Barista Name Lock-In

A barista wrote the wrong name so consistently the customer legally changed his.

610K

VIEWS

3h
OLD

Create coin

Gerald the Frog

A pet frog called Gerald escaped and was found three streets away, in a shoe.

430K

VIEWS

8h
OLD

Buy \$GERALD

SIX ROWS, SEVEN ELEMENTS EACH, NOTHING ELSE. NOTE ROW 3 SITTING ABOVE ROW 4: 850K VIEWS RANKED ABOVE 1.1M. THE LIST IS NOT SORTED BY VIEWS – THE 850K IS TWO HOURS OLD AND STILL CLIMBING, THE 1.1M IS SIX HOURS OLD AND FLAT. THE USER IS NOT TOLD WHY. THEY SEE THE ORDER, AND THE TWO LITTLE LINE GRAPHS QUIETLY AGREE WITH IT.

Two of the six rows say *Create coin*, which is honest: the 3.8-minute figure is measured backwards from coins that exist, and it does not say every viral post gets minted. The overwhelming majority never do. The top row says *3 coins* because the ferry captain already has three — \$FERRY at \$799.6K, \$HORN at \$1.2M, and a third nobody can verify. A search of pump.fun for a single well-known meme returns **306 separate tokens** carrying that name. Which one is the real one is a genuine question with a real answer, and it is a different question from "is this story worth anything." The feed answers the second. Section 03 answers the first, on its own screen, where there is room to show the evidence.

The seven elements, and why each one earns its place

- 1

A picture from the post

The still frame from the video or image. It is how you recognise a story you have already seen scrolling elsewhere, in a fifth of a second, before reading anything.
- 2

A title

"Harbor Last Horn." A short handle so the story can be referred to, searched for, and pasted into a group chat. Deliberately set smaller than the sentence below it.
- 3

Two lines of plain English

The point of the entire screen. You should understand the story without watching the video. Largest, brightest text in the row — it is what you actually judge the trade on.
- 4

How many views

The headline number, big and unambiguous. A true fact about the world: this many people have seen it. Not our estimate of anything, just the count.
- 5

A tiny line graph

The view count over the last hour. Answers the one question a single number cannot: is it still going up, or is it over? Cyan and rising, or grey and flat.
- 6

How old it is

Four hours. Six hours. Read against a six-day arc it is the whole timing story — 850K in two hours is a different animal from 850K in two days.
- 7

One button

Create coin if nobody has made one, Buy \$TICKER if one is clearly the real one, 3 coins → if several exist and picking is its own job. One row, one next move.

What we removed, and where it went

An earlier version of this screen showed the row as a trading terminal: columns headed HEAT, ACCEL, ORG and MEME, a twenty-minute chart of view-rate change, and a footer counting how many candidates our filters had thrown away. Every number on it was real. It still had to go. The test is simple — **would this number still be true if Insider did not exist?** Views, age, who posted it: yes. Heat, acceleration, meme score, how many we filtered: no. Those are our opinion wearing a number's clothes, and putting them on screen hands the judgement back to the person we built the company to make it for.

×

REJECTED — OUR WORKINGS ON THE USER'S SCREEN

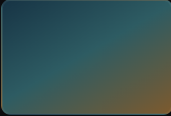
Everything struck through is an internal number. It exists. It never appears.

#	POST	HEAT	REACH	RATE	ACCEL	20M	ORG	MEME	AGE	
1	 He blew the horn one last time and the whole harbour answered. 31 years. @nordkyst_ferge · X · 412K followers	94	2.4M	12.4K/m	3.2x		88	84	4h	Buy \$FERRY
40 of 63 eligible · 12 hidden by gate · meme_score 7 · 0 3 · author cap 2										

✓

SHIPPING — THE SAME STORY, THE SAME INSTANT

Six facts about the world, and one thing to do. Seven, no eighth.



Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

2.4M

VIEWS



4h

OLD

3 coins →

SAME STORY, SAME SECOND, SAME UNDERLYING DATA. THE TOP ROW ASKS THE READER TO WEIGH UP SIX NUMBERS AND DECIDE. THE BOTTOM ROW HAS ALREADY DECIDED, AND SAYS SO BY SITTING AT THE TOP OF THE LIST. NOTE THAT VIEWS, AGE AND THE ACTION SURVIVE THE CUT — THOSE ARE FACTS, NOT OPINIONS. NOTE ALSO THAT THE REJECTED ROW SAYS "BUY \$FERRY" WHEN THERE ARE THREE CANDIDATES AND \$FERRY IS NOT THE BIGGEST — CONFIDENCE IT HAS NOT EARNED.

To be completely clear: **we still calculate every one of those numbers, on every cycle.** They decide the order of the feed, which posts get grouped into one story, and which of the roughly 30,000 coins minted each day are worth a second look — a stream in which the middle coin is worth about \$28 and, in a run of 300 consecutive mints, 174 were repeats of a name already used. Filtering that is most of the work. Reporting that we filtered it is worth nothing to anybody.

INTERNAL NUMBER	WHAT IT DECIDES	WHERE IT LIVES
heat	How high up the feed a story sits right now.	Library → row order only
acceleration	Whether a story is still climbing or has peaked, from two measurements of the same post taken at known times.	Library → row order only
organic score	Whether the attention looks like real people or like a paid push.	Library → included / excluded
meme score	The judgement on whether a coin could plausibly be made out of this at all.	Library → included / excluded
eligibility gate	How many candidates were dropped before the list was built.	Library → never surfaced

THE FEED IS THE OUTPUT OF THIS TABLE, NOT A RENDERING OF IT. IF A ROW IS THIRD RATHER THAN FIRST, THAT IS THE ANSWER — THERE IS NO SCORE TO ARGUE WITH, AND NOTHING FOR THE USER TO RE-DERIVE. THE APP'S JOB IS TO HAVE ALREADY MADE THE JUDGEMENT.

One caveat worth stating plainly, because it comes back in section 06: none of the five rows above is a machine-learned model today. Every threshold in them is a number a person typed, and the qualitative calls are made by Claude reading the post. That is a defensible starting point and a bad ending point — but it changes nothing about this screen. Whether the order comes from typed thresholds or from a trained model, the user sees the same seven things, and the order is still the whole answer.

03. The story page

The main attraction. One story, one web address: what happened, how far it has travelled, every coin anyone made out of it, and what other traders are saying while they hold or don't hold those coins.

Every story gets its own address — insidor.app/s/harbor-last-horn . A trader pastes that link into a group chat and someone who has never heard of us opens it, understands the thing in one sentence, and can act. No account needed to read it. The page is the marketing.

The layout says what we think matters. **Evidence on the left:** the clip, the plain-English two-liner, every post we can find, the view count climbing, and where this sits in the clock. **Discussion on the right**, so a take is never separated from the facts it was made against. **The coins across the bottom**, full width, because that is the part that decides whether somebody makes or loses money.

What we can honestly claim about time

An earlier version of this page said we show you the story before the coin exists. We measured it, and that is false for every coin worth owning. X post IDs carry their own timestamp, so the gap between a post and the coin made from it is exactly computable: the median is **3.8 minutes**, and for coins that later cleared ten million dollars it is **1.1 minutes**. Bots do that. Nobody beats them and we should stop saying we will.

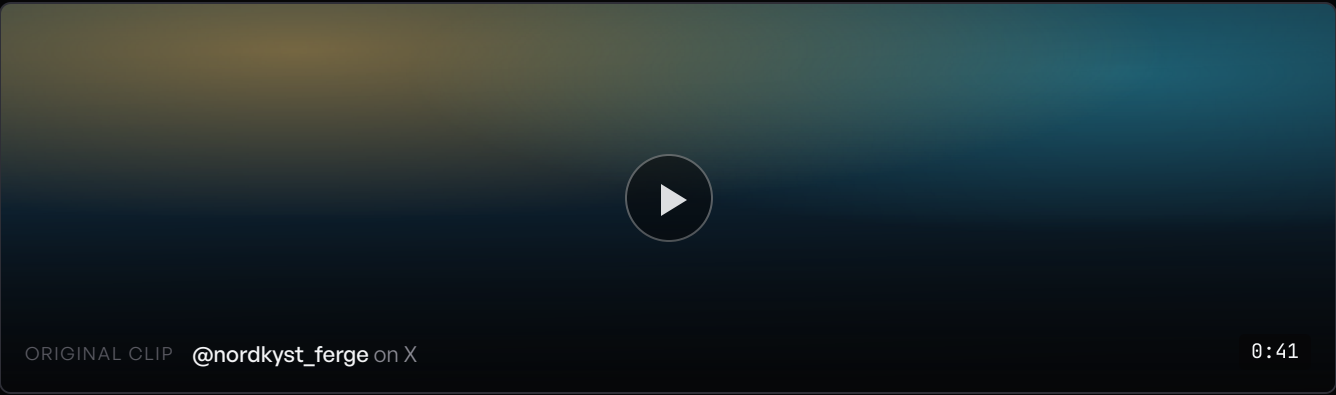
What is still open is much larger. Post to *peak* runs about six days, and of the coins that cleared a million dollars, **none peaked inside the first hour**. So the page does not sell a head start on the mint. It sells the four hours, or four days, between a coin existing and a coin being finished — and the reason to trust it is the view curve on the left, which we recorded ourselves because nobody sells "what did this post look like twenty minutes ago".

Harbor Last Horn

4h12mo1d

Watch

Share



A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

Open the post on X ↗

Open the TikTok clip ↗

Kristiansund, Norway · 09:00 today

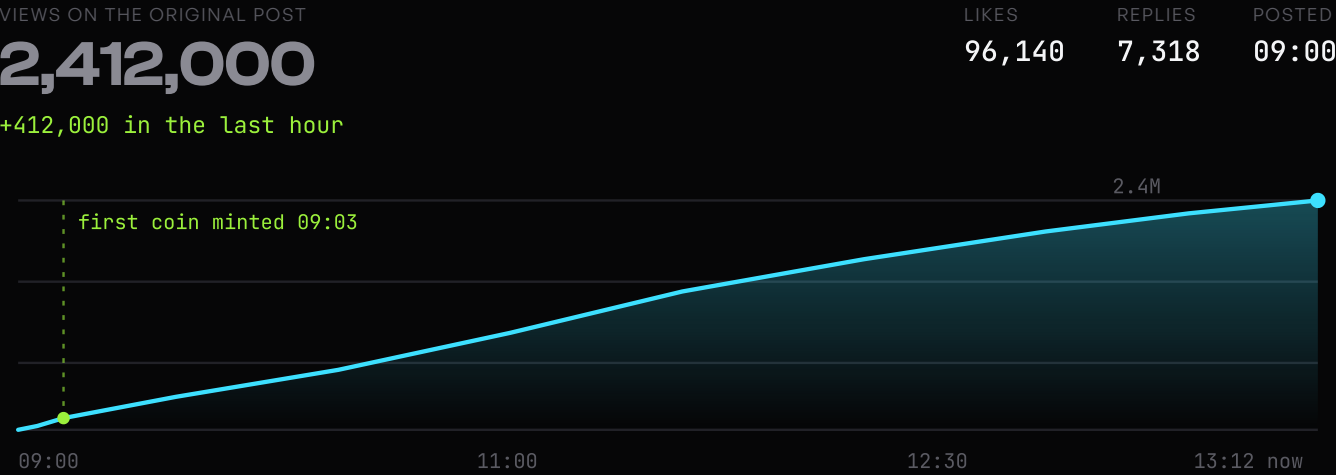
A BIT MORE, FROM THE POSTS

Captain Bjørn Hagen finished his last shift on the Kristiansund route this morning after 41 years. He sounded the horn leaving the quay; six vessels in the harbour answered, then the pilot boat, then the lifeboat station. Filmed from the pier by a passenger and posted forty minutes later.

EVERY POST WE FOUND · 4 · EACH PLATFORM'S OWN NUMBERS

X	@nordkyst_ferge — the original	2,412,000 views · 18,402 reposts · 1,940 quotes	09:00	Open ↗
TikTok	@harbourclips — second angle from the pier	890,000 plays · 21,300 shares · quotes —	09:12	Open ↗
X	@sjofart_no — names the captain, 41 years on the route	214,000 views · 3,110 reposts · 402 quotes	10:26	Open ↗
X	@dailyfeelgood — reposts the clip with English subtitles	78,400 views · 1,204 reposts · 88 quotes	11:41	Open ↗

TikTok has no quote-post, so that box is blank rather than zero. Plays are not views and shares are not reposts; we do not add them together.



Views we recorded ourselves, every few minutes, since 09:04

THE CLOCK ON THIS STORY

09:0009:0313:12

postfirst coinyou are here

day 6 · off the right of

MARKET HISTORY · ALL COINS THAT EVER PASSED \$1M

First coin appeared a median of 1.2 minutes after the post · none peaked inside the first hour · the peak came a median of 6 days after the post.

Discussion14 takes · each one stamped when it was written

Say something about Harbor Last Horn...

WILL BE STAMPED ON: Story at 2.4M viewsSFERRY at \$799.6K

Holding 1.2M FERRY

READ FROM YOUR WALLET · FROZEN WHEN YOU POST · NEVER EDITABLE

@sondre09:44 · 3H 28M AGO

Story at 340K viewsSFERRY at \$61KHolding \$FERRY

Nordic maritime nostalgia travels further than people think — same shape as every "old man's last shift" clip that ran for days. The boats answering back is what makes it a meme and not a news story. Took \$FERRY because it was first and the name is the obvious one.

▲12812 repliesShare

@meridian11:20 · 1H 52M AGO

Story at 1.1M viewsSHORN at \$980KSold \$HORN

\$HORN has the liquidity but two top wallets have been in since minute four and one just trimmed. I took mine off at \$980K. Not a call on the story — the story is fine — a call on who is sitting in that particular coin.

▲8621 repliesShare

@ke1p13:06 · 6M AGO

Story at 2.4M viewsSFERRY at \$782KNo position

Late here and saying so. Wholesome clips top out fast unless a large English account picks them up, and that has not happened yet. Watching, not buying.

▲314 repliesShare

11 more takes · oldest stamped at 41K views, 09:12

Coins made from this storya name search on pump.fun returns 306 coins using this name or artwork

See that search ↗

First — made before any other

Biggest — most money in it now

Matched — we tied it to this story

Unverified — we could not, so no buy button

COIN	MINTED	NAMES ITS SOURCE	1H	MARKET CAP · HIGHEST	LIQUIDITY	HOLDERS	
FIRST \$FERRY Harbor Ferry	09:03 3 min after the post	Links this X post	+18.4%	\$799.6K peak \$812.4K	\$88.4K	1,204	Buy \$FERRY
BIGGEST				\$1.20M			

\$HORN Last Horn	MATCHED 09:04 1h 41m after the post	Names nothing	-4.1%	peak \$1.40M · 11:44	\$141.2K	2,867	Buy \$HORN
\$CAPTAIN Captain Bjørn	10:41 1h 41m after the post	Links the TikTok	+6.1%	\$128.4K peak \$141.0K	\$22.6K	511	Buy \$CAPTAIN
\$BOAT Boat Horn	UNVERIFIED 12:58 3h 58m after the post	Names nothing	-9.4%	\$61.4K peak \$88.2K	\$6.2K	87	No buy button. Open on Solscan ↗
Why \$BOAT has no button. Its picture is a boat and its name is a horn, but it names no source and it appeared four hours late. It might belong to this story; it might be riding a different one. We would rather list it and say so than hide it, and rather leave the button off than help you buy something we cannot stand behind. The explorer link still works.							
Only about one coin in twenty above \$50K names the post it came from, so most of the time this column reads Names nothing and the match is a judgement. That is why the list shows every candidate rather than picking one for you.							

Every number here is a fact about the world — views, plays, mint times, market cap, liquidity, holders. Our judgement appears as words and as which rows exist, never as a score printed beside them.

The coin list is the product

One meme spawns a swarm. A pump.fun search for "chill guy" returns **306 distinct tokens**. Roughly thirty thousand coins are minted every day, the median one is worth twenty-eight dollars, and in a run of three hundred consecutive mints more than half reused a name already taken — one name was minted twenty-one times. Detecting the story is one problem. Working out *which of the 306 is the real one* is a completely separate problem, and it is the one that actually costs a user money.

So the list shows four states and never fewer. **First** is the earliest coin made from this story. **Biggest** holds the most money right now. They are usually different coins and which one you want depends on how you trade. **Matched** is everything else we are confident belongs here. **Unverified** is a coin we suspect belongs and cannot prove — and it gets no buy button at all, not a greyed-out one.

Each row also carries the highest market cap that coin has ever reached, beside the one it has now. Among the thousand best-performing coins ever minted, the median peaked near **\$546,000 and sits today near \$3,000**. They do not fail to get traction — they get it and hand back over ninety-nine percent of it. A current market cap shown alone is therefore half a fact, and on this page \$HORN's "\$1.20M, peak \$1.40M" tells a user something that "\$1.20M" alone conceals.

✗ What a disabled button says \$BOAT Boat Horn · \$61.4K Buy \$BOAT A greyed-out button is still a button. It reads as "we are holding this back from you" — a lock, a payroll, a thing to be unlocked. Users click greyed buttons. Some of them will find another way to buy it and feel they were told to.	✓ What we do instead \$BOAT Boat Horn · \$61.4K No buy button. Open on Solscan ↗ No button exists. The row is still there, the numbers are still real, the explorer link still works. We are not hiding the coin — we are declining to make a claim we cannot stand behind, and the absence says so more clearly than any tooltip.
--	--

The founder's rule, unchanged: "A greyed-out button looks like something we're holding back; no button at all is us not making a claim we can't stand behind."

Everyone's numbers in their own words

The evidence column lists each post with the platform's own vocabulary — X has views, reposts and quotes; TikTok has plays and shares and no quote-post at all. This is not fussiness. Our own code currently pours TikTok's play count into a column called views, its share count into one called reposts, and writes a zero into quotes because TikTok has no equivalent. Quotes are the signal the whole product rests on — people reproducing a story in their own words — so for every TikTok post that signal silently reads as an honest zero. On this page we print a dash instead, and we never add plays to views to make one bigger number.

The right-hand column, and why nobody else can build it

A take is worthless without knowing when it was made and what the person saying it owned. So both are stamped on at the moment of posting and can never be edited: how big the story was, what the coin was worth, and whether the writer was holding, had sold, or had nothing on the table — read from the connected wallet. Somebody who called this at 340,000 views reads very differently from somebody who turned up at 2.4 million. Neither is disqualifying; both are things you would want to know.

Social apps know what people say. Trading apps know what people own. Insider is the only place the take and the wallet arrive together, on the same story, at the same second — so a record of who was right, when, and with their own money on it writes itself. A competitor starts with none of that history and cannot buy it later.

What is deliberately not on this page

No score, no heat number, no acceleration figure, no confidence percentage beside \$BOAT, no count of the coins we ruled out. The 306 is on screen because pump.fun would return it whether or not we existed; our own arithmetic is not, because it would not. The user gets our judgement — which story, which coins, which one has a button — and never the workings.

04. Create a coin

An earlier version of this section promised the user would be first. The timing data says that is false for every coin worth being first on. So this screen keeps the same shape and loses the boast: it is the smaller of the two doors, it tells the truth about the odds on the screen itself, and — unlike the build we have today — it actually creates a coin.

The claim we used to make, and why it is dead

X post IDs contain their own timestamp, so the gap between a post going up and a coin being minted from it is computable, not guessable. The median gap is **3.8 minutes**, and it shortens as the coin does better: **1.2 minutes** above a \$1M peak, **1.1 minutes** above \$10M, with 88% of those minted inside the first hour. Bots watch the same firehose and are faster than any interface with a human in it. Chill Guy — the famous thirteen-month gap — was an archived tweet re-minted years later, and every case like it peaked between \$159K and \$1.8M. The slow regime is the losing regime.

The second half of that measurement is the good news. Post-to-*peak* is about six days, and **zero percent** of coins clearing \$1M peaked inside their first hour. The mint window shuts in minutes; the price window stays open for days. That is why buying (section 03) is the main door and this is the side door.



Two different clocks, drawn on one line. Creating is a race we lose in the first ninety seconds. Trading is a race that runs for a week. The product should be weighted accordingly.

So what is this screen actually for

Three honest cases. The story genuinely has no coin — most viral posts never get minted. The user is close to the story and wants to be the one who mints it, for reasons that have nothing to do with beating a bot. Or — the real volume — they have seen the tickers we found, decided none is the real one, and want their own. A pump.fun search for "chill guy" returns 306 distinct tokens. "No coin" almost never means no coin; it means *no coin we can match to this story*, and the screen should say so.

The example is **"Paid to Move to Norway"** — a Norwegian town paying €25,000 to people who relocate. 1.1M views, an hour old, nothing trading under any ticker we can tie to it.

The create sheet

One screen. The story sits locked at the top so it is always obvious what this coin is *of*. Name, ticker and image arrive pre-filled — written an hour earlier as a by-product of the AI reader judging the post, so naming costs nothing extra and is done long before a human turns up. Everything is editable; nothing requires research.

Create the coin

A Norwegian town is paying people €25,000 to move there.

1.1M views · 1h old · @nordic_notes ·

NO COIN FOUND

NAME

Edit

Paid To Move To Norway

TICKER

Edit

\$NORWAY

✔ NOTHING TRADING

\$NORWAY

\$MOVE

\$FJORD

\$25K · 2 live

Checked against pump.fun 4 seconds ago. Solana does not reserve symbols — two coins already trade as \$25K, and anyone can mint a third \$NORWAY after you.

IMAGE

Change

FROM THE POST

+

BUY SOME YOURSELF FIRST — OPTIONAL

0.25

0.5

1 SOL

2

Custom

Buys 34.61M tokens — 3.2% of supply — at the starting price.

Your first buy

1.0000 SOL

pump.fun launch

0.0200 SOL

Solana network fee

0.0002 SOL

Insidor charge

NONE

TOTAL FROM YOUR WALLET

1.0202 SOL

≈ £155 · you hold 3.4118 SOL

WHAT USUALLY HAPPENS TO A NEW COIN

Minted on pump.fun yesterday

29,847

Median value of one, fresh

\$28

Share that ever clear \$10K

1 in 300

Of the 1,050 biggest ever: peak, then today

\$546K → \$3K

AFTERWARDS, ON EVERY TRADE OF THIS COIN

You, as the creator

0.05%

Insidor

0.02%

Review in wallet

Insidor never holds your money or your keys. We prepare the transaction; you approve it in your own wallet on the next screen.

"NO COIN FOUND", NOT "NO COIN EXISTS"

Only about 5% of coins over \$50K declare which post they came from, so matching is inference. The badge should claim what we actually know and nothing more.

NAME AND TICKER ARRIVE FILLED IN

Written an hour ago when the AI reader judged the post. *Edit* replaces them; the alternatives are one tap away.

AVAILABILITY IS A LIVE LOOKUP, AND IT IS NOT A RESERVATION

Solana enforces no uniqueness on symbols. "Nothing trading" is true right now and stops being true the moment someone else mints. Saying "you'd be first" would be a lie with a shelf life.

THE IMAGE IS A STILL FROM THE POST

Three alternate frames plus an upload slot. Most users accept the first one, which is the point.

THE FIRST BUY IS OPTIONAL AND SPECIFIC

It says what 1 SOL gets and what share of supply that is. Turning the toggle off drops the total to 0.0202 SOL.

THE ODDS GO ON THE SCREEN

Every figure in that amber block is a fact about pump.fun that would be true if Insidor did not exist. None of it is our scoring. It is the least we can do on a screen that is about to take £155.

ONE TOTAL, IN POUNDS

Crypto interfaces habitually show a number the user has to convert in their head. We show SOL and sterling, and we say plainly that we charge nothing at this step.

HOW WE ACTUALLY GET PAID

A cut of trades afterwards, smaller than the creator's. Stated on the screen where it applies rather than buried in terms.

Test applied to every number here: would it still be true if Insidor did not exist? Views, age, who posted, cost, fee split, share of supply, the pump.fun population statistics — all yes. The judgement that put this story in front of the user stays behind the curtain.

The ticker check has four answers, not two

The check runs while the user is still looking at the field — discovering a collision after a signature is a terrible experience. It is a network lookup against pump.fun, not a look at our own memory, and its honest vocabulary is "nothing trading under this", never "yours".

WHILE WE CHECK

\$NORWAY

Half a second, usually. [Review in wallet](#) waits rather than letting the user run ahead of the answer.

NOTHING TRADING

\$NORWAY

✓ CLEAR

Carry on. Not a reservation — the check is a photograph, not a lock.

SOMETHING ALREADY TRADES

\$25K

2 LIVE

Two coins already trade as \$25K — \$84K and \$2.1K. We show both, because buying one may beat minting a third.

LOOKUP FAILED

\$NORWAY

NO ANSWER

We could not reach pump.fun. Retry, or continue knowing we did not check. Never silently pretend it came back clear.

The fourth card is the one the current build lacks entirely — it cannot fail, because it never asks anyone. A check that cannot fail is not a check.

Signing it

This is where the safety promise is kept. Insidor assembles the transaction and hands it to the user's wallet — Privy, so a specialist handles custody — which shows what is about to happen and asks for approval. We never take custody and never see a private key. Reject it and nothing is spent. The app already does exactly this for buying: `api/swap.js` builds a Jupiter swap and the browser signs it. Creating should use the same road.

Approve transaction

YOUR WALLET

ACTION	Create token + buy
PROGRAM	pump.fun
TOKEN	Paid To Move To Norway (\$NORWAY)
YOU SEND	1.0202 SOL
YOU RECEIVE	34,612,903 \$NORWAY
EXPIRES	in 48 seconds
REQUESTED BY	insidor.app

Reject

Approve

Signed on your device. Insidor cannot move these funds and cannot sign on your behalf.

✓

\$NORWAY is live

CONFIRMED 14:22:07 · SLOT 298,441,062

YOU HOLD	TOKENS	MARKET CAP	HOLDERS
3.2%	34.61M	\$5.9K	1

CONTRACT ADDRESS

6xKqR3vTn8mLbY2sHwEeA9dPgUcJf1ZoNrQ4iSVXpump

read back from the confirmed transaction · tx 5Jm...9tQ · verify on Solscan

Share the story

View the coin

pump.fun

Copy address

31 MINUTES LATER

Holders	214
Market cap	\$88.4K
Your creator earnings	£6.20

One line does the heavy lifting: the address is read back from the confirmed transaction. It is the mint account the chain assigned, not a string we composed — and the tx signature is shown beside it so the user can check us.

Between tapping Approve and a coin existing there are four outcomes, and the build has one. A blockhash expires in about a minute; transactions get dropped; wallets get rejected; balances come up short once the fee lands. None of these should leave a coin in the interface that does not exist on the chain.

● Sent

Signature accepted, waiting for the network. We poll for confirmation and show the elapsed seconds honestly.

TYPICALLY 1-3 SECONDS

● Confirmed

The mint account exists. Only now do we write the coin into the story page and show the address.

THE ONLY PATH TO "IS LIVE"

● Rejected

The user pressed Reject, or the window expired. Nothing spent, sheet returns with every field still filled in.

COSTS NOTHING · RETRY IN ONE TAP

● Failed on chain

Dropped, or short of SOL once fees landed. We say which, and we never claim a coin that did not get created.

NETWORK FEE MAY STILL BE SPENT

Four states, one of which is success. The build ships the success state only, and reaches it unconditionally.

What the build does today — said plainly

The create flow in the current build does not create anything. Pressing the button invents a contract address with a small random number generator, inserts a token into an in-memory array so it appears in the feed, and tells the user their coin is live. No wallet opens. No transaction is built. Nothing touches Solana. "Copy CA" then puts the invented string on the user's clipboard.

SITE/INDEX.HTML – THE ADDRESS IS GENERATED, NOT RECEIVED

LINE 291 • GENCA()

```
function genCA(s){ // 44 characters that look like a Solana address
  let x = s*9301+49297;
  for(let i=0;i<44;i++){ x=(x*9301+49297)%233280; o+=h[x%h.length]; }
  return o; }
                                ← a linear congruential PRNG
```

LINE 672 • DEPLOYFILED()

```
const ca = genCA(tick.length*37 + tick.charCodeAt(0));
TOKENS.unshift(nt); // coin now "exists" – in a JS array
sheet.innerHTML = `<div class="done-h">${tick} is live</div>`;
$('#dcopy').onclick = () => navigator.clipboard.writeText(ca);
```

Two more things in the same flow are invented rather than fetched. The availability check at line 653 searches the app's own arrays, then prints “*SNORWAY is free on Solana – you'd be first to deploy it*” – a sentence about the blockchain produced without contacting it. And the cost line is **0.52 + firstBuy** : a hardcoded 0.52 SOL launch fee, roughly twenty-six times the real pump.fun charge of about 0.02 SOL.

The uncomfortable part is that the machinery already exists. **api/swap.js** builds a real transaction and the token page has the browser sign it through Privy. Buying is real; creating took the shortcut, and nothing in plain JavaScript objected – the same absence of shape that let TikTok play counts get written into a column named **views** . This is the one bug in the build that could cost a user money on the strength of a number we made up.

Shown here not to embarrass the build but to size the work: the create path is a mockup wearing the interface, and everything above in this section is what replacing it looks like.

Build it or rent it

PIECE	WHO DOES IT	STATUS TODAY
Knowing this story has no coin we can match	Insidor	Real — the clustering and matching already run.
Suggested name, ticker and image	Insidor, via Claude	Real — a by-product of judging the post an hour earlier.
Checking what already trades under a ticker	pump.fun	Fake — searches our own arrays, answers about Solana.
Building the create transaction	pump.fun program, assembled by us	Missing — no endpoint exists. Roughly three days' work.
Holding the wallet and signing	Privy, on the user's device	Real for buying; the create path simply never calls it.
Confirming and reading back the mint address	Solana RPC	Fake — a pseudo-random string, copied to the clipboard.

Same rule as everywhere else: build the detection, rent the plumbing. What is genuinely ours on this screen is knowing there was something worth minting. What is missing is the two hundred lines that turn that knowledge into a transaction the chain will accept.

05. Trading, watchlist, search and trending

Four supporting features. None of them is the reason someone opens Insidor, but without them the person who spots a story early has to leave to act on it, forgets to come back, buys the wrong one of three hundred coins with the same name, or can only see what we choose to push at them.

Trading — buying built into the page, and the fee said out loud

When a narrative already has a coin, the buy box sits on the same page as the story. No copying a contract address into another app, no second tab, no losing the thing you were reading. We take a small fee on every trade that goes through us, and the whole cost is itemised in the order before you confirm — ours, the exchange's, and the network's, added up into one number and one percentage. Nothing hidden inside the price.

We do not build the trading engine. Routing an order, getting a fair price, handling a failed transaction without losing someone's money is hard, risky, and already solved by firms who do only that. **The plan is to route through Axiom or Photon, pending confirmation that they permit it: research found neither publishes a public developer interface, so the integration is not yet proven.** The confirmed fallback is Jupiter, which is openly documented and pays us an integrator fee — where Photon charges the user 1% and pays us nothing.

Two facts changed this panel. The first: a search for one meme on pump.fun returned **306 different tokens**. Choosing which of them is the real one is a separate problem from spotting the story, and it is the problem that loses people money. So the panel names exactly what you are about to buy — ticker, full contract address, mint time — above the amount box, not buried in a tooltip. The second: among the 1,050 best-performing coins ever minted, the median one peaked near **\$546,000 and now sits near \$3,000**. They do not fail to get traction; they get it and give back over 99%. A current market cap shown on its own is therefore a half-truth, so the panel prints the highest market cap the coin has reached and when, right next to today's.

NARRATIVE

Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

VIEWS

AGE

POSTS

COINS

2.4M

4h

2

3

\$FERRY

FIRST

\$799.6K

+18.4% 1h · peak \$812.4K

\$HORN

BIGGEST

\$1.2M

-4.1% 1h · peak \$1.4M

\$BOAT

UNVERIFIED

\$61.4K

no source post declared

09:31

11:15

peak \$812.4K

now

MARKET CAP

HIGHEST SO FAR

LIQUIDITY

HOLDERS

CREATED

\$799.6K

\$812.4K

\$84.2K

1,204

09:31

4 min after the post

RECENT TRADES

last 90 seconds · 1,204 holders

TIME	SIDE	SOL	TOKENS	WALLET
13:31:02	BUY	1.20	259,330	7Bv...q4T
13:30:47	SELL	0.35	75,638	4mK...zR9
13:30:21	BUY	0.10	21,611	Hs2...8Lc
13:29:58	BUY	2.00	432,216	Qw9...3Xd
13:29:33	SELL	0.75	162,081	5nT...bV1

Every number on this panel is a fact you could check somewhere else: price, market cap, the highest market cap it has reached, liquidity, holders, mint time, the last five trades, and each fee by name. The contract address is shown in full because "\$FERRY" is not an identity – hundreds of tokens can carry the same three letters, and only the address says which one your money is going into.

Watchlist — the notification is the feature

Saving a narrative is easy to build and easy to ignore. The part that matters is what happens after: if a story you saved gets its *first* coin, we tell you within seconds. That is the whole retention mechanic. Rather than hoping the user remembers to check back, we open the app for them at the one moment there is something to do.

The measured timing says this alert will usually fire fast. Across coins we could date precisely, the median gap between the post and the first mint was **3.8 minutes**, and for coins that went on to clear \$10M it was about **one minute**. Save a story that has no coin and the alert often lands before you have put the phone down. It is worth sending because the mint is not the finish line: **no coin that cleared \$1M reached its peak within an hour**, and the typical gap from post to peak is around six days. A four-minute-old coin is early, not over.

Everything else stays inside the app. If we send noise the user turns notifications off, and you only get to kill this mechanic once.

Buy

Sell

\$FERRY

FERRY HORN

9xQe4Rm2vK8sD1tYbN3gLpXwH7aZcU6fQe2RmTkPpump

made 09:31 · 4h old

0.50

SOL

≈ \$86.40 · balance 4.21 SOL

0.1

0.5

1

2

Max

YOU RECEIVE

106,378 \$FERRY

Price per token

\$0.0007996

Price impact

0.31%

Max slippage

1.0%

WHAT THIS TRADE COSTS

Exchange fee (1.0%)

\$0.86

Insidor fee (0.5%)

\$0.43

Solana network fee

\$0.02

Priority fee

\$0.04

Total fees

\$1.35 · 1.56%

Selling later costs about the same again

≈ 3.1% round trip

Buy \$FERRY · \$86.40

\$FERRY is 4h old. Its highest market cap so far was \$812.4K, reached at 11:44. It is 1.6% below that now.

Routed to Axiom · wallet by Privy

Jupiter used if routing is unavailable

1 · SAVED EARLIER · 11:02

Watchlist

4 SAVED

Paid to Move to Norway

1.1M views · 1h · no coin yet

ALERTS ON

Barista Name Lock-In

610K views · 3h · no coin yet

ALERTS ON

Harbor Last Horn

2.4M views · 4h · 3 coins

ALERTS ON

Plastic Plant Grandma

850K views · 2h · \$PLANT \$562.8K

ALERTS OFF

2 · PHONE LOCKED · 11:06

11:06

Tuesday 14 October

INSIDOR

now

First coin on a story you saved

“Paid to Move to Norway” now has a coin — \$MOVE, made 2 minutes ago.

Open Watchlist

3 · TAPPED · 11:06

Paid to Move to Norway

A Norwegian town is paying people €25,000 to move there.

VIEWS

1.1M

+184K last hour

AGE

1h

posted 10:02

POSTS

2

X · TikTok

NEW — FIRST COIN

\$MOVE

made 11:04 · 2 min ago

\$18.7K

41 holders

Buy \$MOVE

View the two posts

WAKES THE PHONE

The first coin on a saved story · once only, never again for that story

A new post joining a saved story · at most one an hour

NEVER WAKES THE PHONE

The second, third and 306th coin on the same story

Price moves, market cap milestones, daily digests, “you might like”

The notification carries the two-liner, not a ticker on its own — the user saved a story, so remind them which story. Tapping it lands on the coin with a buy button already there: two taps from lock screen to trade. The rules under the mockup are the whole notification policy; there is nothing else we are allowed to send.

Search — one box, and it stops duplicates

One field takes three kinds of input: a keyword (“ferry”), a ticker (\$FERRY), or a coin address pasted from anywhere. It returns both sides at once — the stories we are tracking and the coins that exist — and its most useful job is unglamorous. A pump.fun search for one popular meme returns **306 distinct tokens** carrying that name. So this screen answers two questions with the same list: *is there already a coin for this, before I make another one*, and *which of these is the one people are actually in*.

It cannot lean on metadata to do it. Only about **5% of coins above \$50K declare the post they came from**, so most of the pile is anonymous by construction. The screen therefore shows what each coin itself says: address, mint time, liquidity, and whether it names a source post or not. Coins that declare nothing are still listed — hiding them would be its own kind of lie — but the near-worthless tail is folded into a single line. The median freshly minted coin is worth **\$28**, and a list that presents those as peers of a \$799.6K coin is misleading by layout alone.

🔍 ferry|

keyword · \$ticker · coin address

NARRATIVES

2 results

Harbor Last Horn

2.4M views · 4h · 2 posts · 3 coins

2.4M views

Ferry Cat Stowaway

180K views · 9h · 1 post · no coin

180K views

COINS NAMED FERRY OR SIMILAR

14 on chain · 3 with liquidity above \$1K

\$FERRY · Ferry Horn

9xQe4Rm2...pump · made 09:31 · liquidity \$84.2K

Names a source post — @nordkyst_ferge, 09:27

\$799.6K

+18.4% 1h

\$HORN · Last Horn

C2fH8vTq...pump · made 09:38 · liquidity \$141.2K

Names a source post — @nordkyst_ferge, 09:27

\$1.2M

-4.1% 1h

\$BOAT · Boat Horn

Ak7Ln1Wd...pump · made 11:52 · liquidity \$6.2K

No source post declared

\$61.4K

-9.4% 1h

+11

11 more tokens named **FERRY**, **FERRY**Y or **FERRY HORN**, minted between 09:33 and 12:41. Combined liquidity **\$418**. None declares a source post.

Show all

● Coins for this story already exist.

The earliest was minted at 09:31, four minutes after the post. Check these before you make another one.

Addresses are truncated on screen but are the real thing, and every line is checkable on any explorer: mint time, liquidity, whether the coin's own metadata names a post. The “+11” row is the honest shape of the market — most tokens sharing a name are worth nothing, and saying so in one line beats eleven rows that look like choices.

Trending — ordered by things the user picked

The feed shows what we think is worth seeing right now. Trending lets someone ignore that and look for themselves. It is the release valve for anyone who does not want to be led, and it is deliberately dumb: the only orderings on offer are **views** and **age** for stories, and **market cap** and **age** for coins. No heat, no score, no "top picks". Every one of those sorts is something the user can verify against the outside world, which means a rank here is never a claim of ours smuggled in as a number.

The coins column carries a second figure: the highest market cap the coin ever reached. Among the best 1,050 coins ever minted the median peaked near \$546,000 and now sits near \$3,000, so a list of "biggest coins" that hides the peak is a list of survivors mid-collapse presented as winners. Showing both lets someone see at a glance whether they are looking at something still climbing or something already handing money back.

Top narratives				VIEWSAGE	
01		Harbor Last Horn 4h · 3 coins		2.4M	views
02		Paid to Move to Norway 1h · 1 coin, 2 min old		1.1M	views
03		Grandmaster vs Pigeon 6h · 1 coin		1.1M	views
04		Plastic Plant Grandma 2h · 1 coin		850K	views
05		Barista Name Lock-In 3h · NO COIN YET		610K	views
Ordered by views , highest first. Switch to age and the same rows reorder by when the first post went up — nothing else moves a row.					

Top coins				MARKET CAPAGE	
01		\$HORN Harbor Last Horn · 3h		\$1.2M	peak \$1.4M · 12:10
02		\$FERRY Harbor Last Horn · 4h		\$799.6K	peak \$812.4K · 11:44
03		\$PIGEON Grandmaster vs Pigeon · 5h		\$753.5K	peak \$1.9M · 08:20
04		\$PLANT Plastic Plant Grandma · 2h		\$562.8K	peak \$571.0K · 12:38
05		\$BOAT Harbor Last Horn · 1h		\$61.4K	peak \$88.2K · 12:04
Peak is the highest market cap the coin has reached, and the time it got there. \$PIGEON is 60% below its own high.					

Two lists, two sort controls each, nothing else. Rank 03 is the point of the peak column: by market cap it looks like the third-best coin on the board, and by its own history it has already given back 60%. Both numbers are facts about the world; the user decides what they mean.

All four lean on things we rent rather than build: Privy holds the wallet, Axiom or Photon — or Jupiter — executes the trade, DexScreener supplies price and liquidity, the chain supplies mint times and holders, and the phone delivers the alert. What is ours is the moment each one fires, and the discipline about what reaches the glass. Not one number on these four screens is an Insidor number: views, ages, market caps, peaks, liquidity, holder counts, fees, trades, mint times, contract addresses — every one of them would still be true if Insidor had never been built.

PART TWO

The machinery

How a post is found, grouped, matched to a coin, and how the whole thing gets smarter. None of this is ever visible to a user.

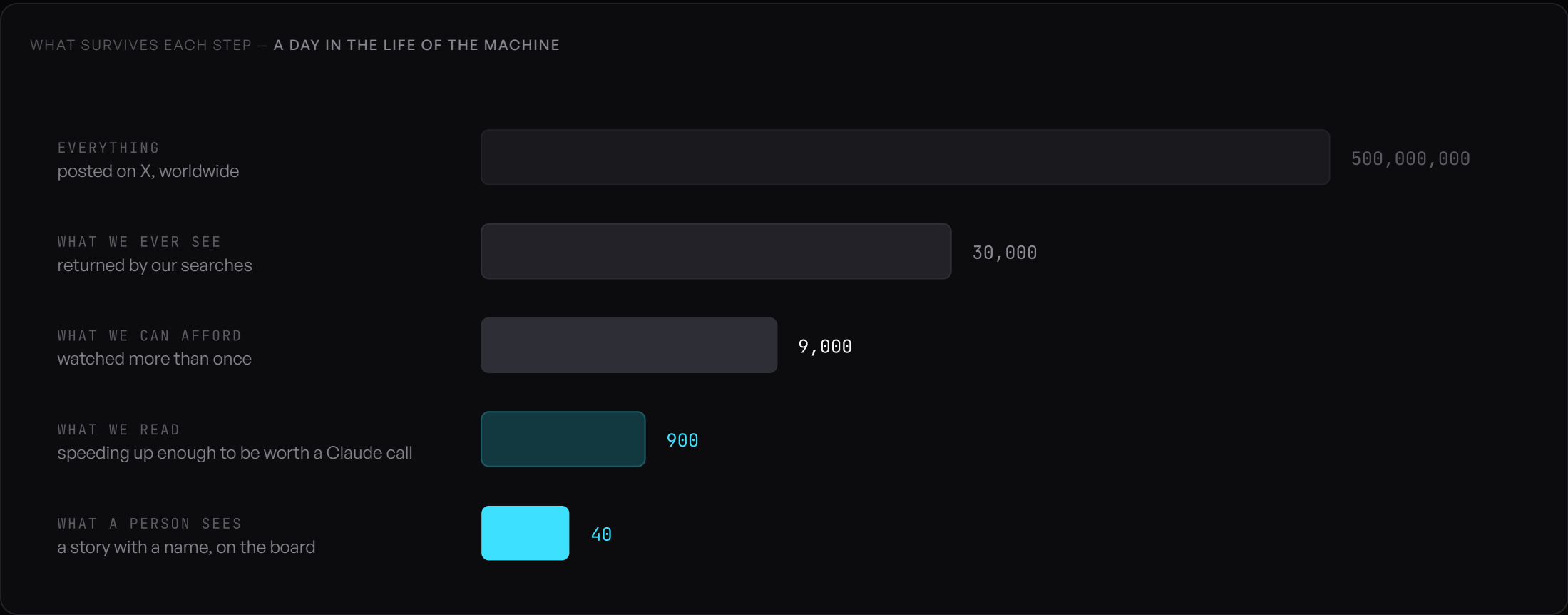
06	How we find viral posts
07	How posts become stories
08	How stories become coins
09	How the system learns
10	What we store, and what it costs

06. How we find viral posts

About half a billion posts go up on X every day. We can pay to look at roughly thirty thousand of them, and to look twice at about nine thousand. Everything in this section is about how the machine chooses which nine thousand — and why waiting for a post to get big is the one strategy guaranteed to be too late.

Reading a post costs money — fifteen hundredths of a cent. That sounds like nothing until you notice that a post worth following has to be read eight or nine times, because you cannot tell whether something is speeding up from a single look. So a budget of thirty-three thousand reads a day does not buy thirty-three thousand posts. It buys about nine thousand watched properly, out of half a billion. One post in fifty thousand. Every other post is a decision made without ever seeing what happened next.

Which is the honest way to describe this part of Insidor: not a detector, a *chooser*. The funnel below is all of it, and the brutal steps are the first two, not the last two.



Each step has to throw away at least 90% of what it received, or it is not a step — it is a rename. The one place the current build fails that test is grouping: 1,200 posts in, 1,289 stories out. That is section 08's problem.

Why "show me posts over 100K views" is structurally too late

The obvious build is a search: give me every post above a hundred thousand views. It is obvious, it is cheap, and it cannot work — for three separate reasons, any one of which is fatal.

A threshold is a queue. The number is public, anyone can run that query, and the people running it fastest are not people. Using the timestamp X hides inside every post's ID, we measured the gap between a post going up and a coin being minted from it: median **3.8 minutes**, and it gets *shorter* the better the coin does — 1.2 minutes for coins that peaked above a million dollars. A post takes most of an hour to reach a hundred thousand views. By the time the threshold fires, the coin has existed for fifty minutes.

A threshold is a level, and a level is history. Five hundred thousand views is a statement about what already happened, not about whether anything is still happening. Two posts can sit on the same number for opposite reasons, and the chart below is that fact drawn out.

A threshold means different things to different accounts. A hundred thousand views on an account with two million followers is a quiet afternoon. The same number on an account with four thousand followers is a structural event — it means the post left the building.

Virality begins at the moment a post escapes its author's own followers. Not when it gets big — when the engagement starts arriving from strangers.

Everything before that point is an account talking to the audience it already has, which is predictable from follower count alone. The first quote or repost from somebody with no connection to the author is the earliest evidence that anything unusual is happening, and it arrives minutes before the view count says anything at all.

This is not a metaphor about how the internet feels. It is how X is built. Posts reach you through two pipes: one seeded from the accounts you follow, and one that searches the whole site for things it thinks you would like regardless of who wrote them. The second pipe is where reach comes from, and it does *not* wait for a post to accumulate engagement before it starts distributing. Being early is technically possible rather than merely aspirational.



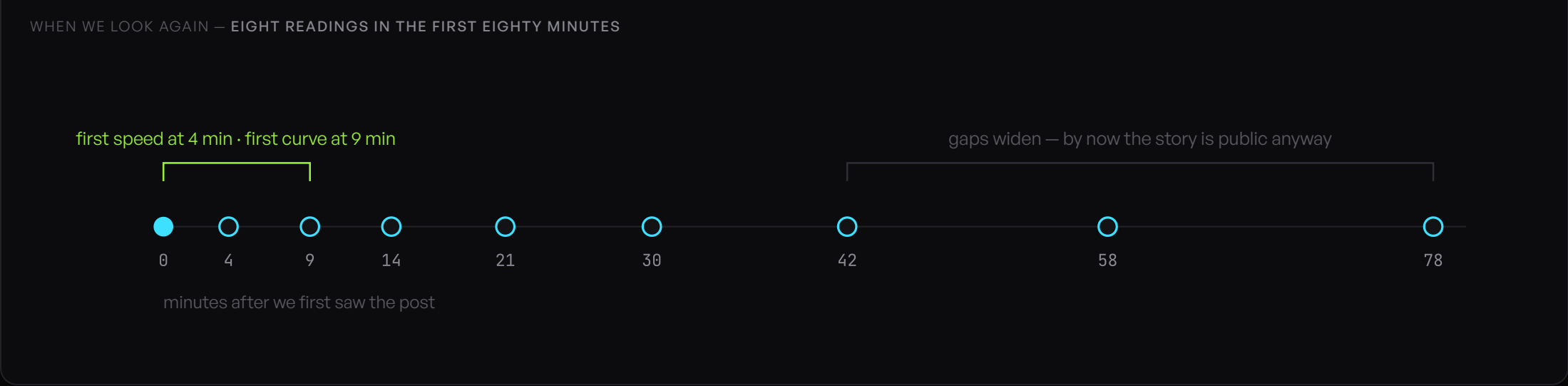
X does not publish a follower / non-follower split on its view count — it is one aggregate number. So the crossing is measured by proxy, from things that do ship free with every post.

Four proxies do most of the work, and the important thing about all four is that they cost nothing extra. They are already sitting in the data we paid for when we read the post the first time.

WHAT WE READ	WHY IT MEANS "STRANGERS"	READABLE FROM	COST
Quotes versus reposts	A repost is one more viewer. A quote is one more <i>version</i> , written by a different person, which X's feed treats as a brand-new item with its own author and its own audience. A thousand reposts collapse into one thing in the feed; a thousand quotes stay a thousand things. This is "people are making their own" expressed in the platform's own plumbing.	the first quote	free
Reach against the author's own ceiling	Views compared with what this specific account's own audience has produced on its own past posts. Four thousand views on a four-thousand-follower account is not noise, it is a ceiling being broken.	first reading	free
Distinct unconnected authors	How many separate people, with no follow relationship to the original, have posted their own version. Not how many posts — how many <i>people</i> . That is the difference between a meme and a video everyone watched.	second version	free
Work per view	Replies and quotes per thousand views. Needing fewer impressions to provoke one reproduction is the reach-normalised version of "this is good", and it keeps meaning the same thing when the platform changes how it distributes.	first reading	free

The snapshot grid

Once a post is admitted, we go back and look at it again on a fixed schedule — dense at the start, thinning out as it ages, because all the interesting change happens in the first half hour.



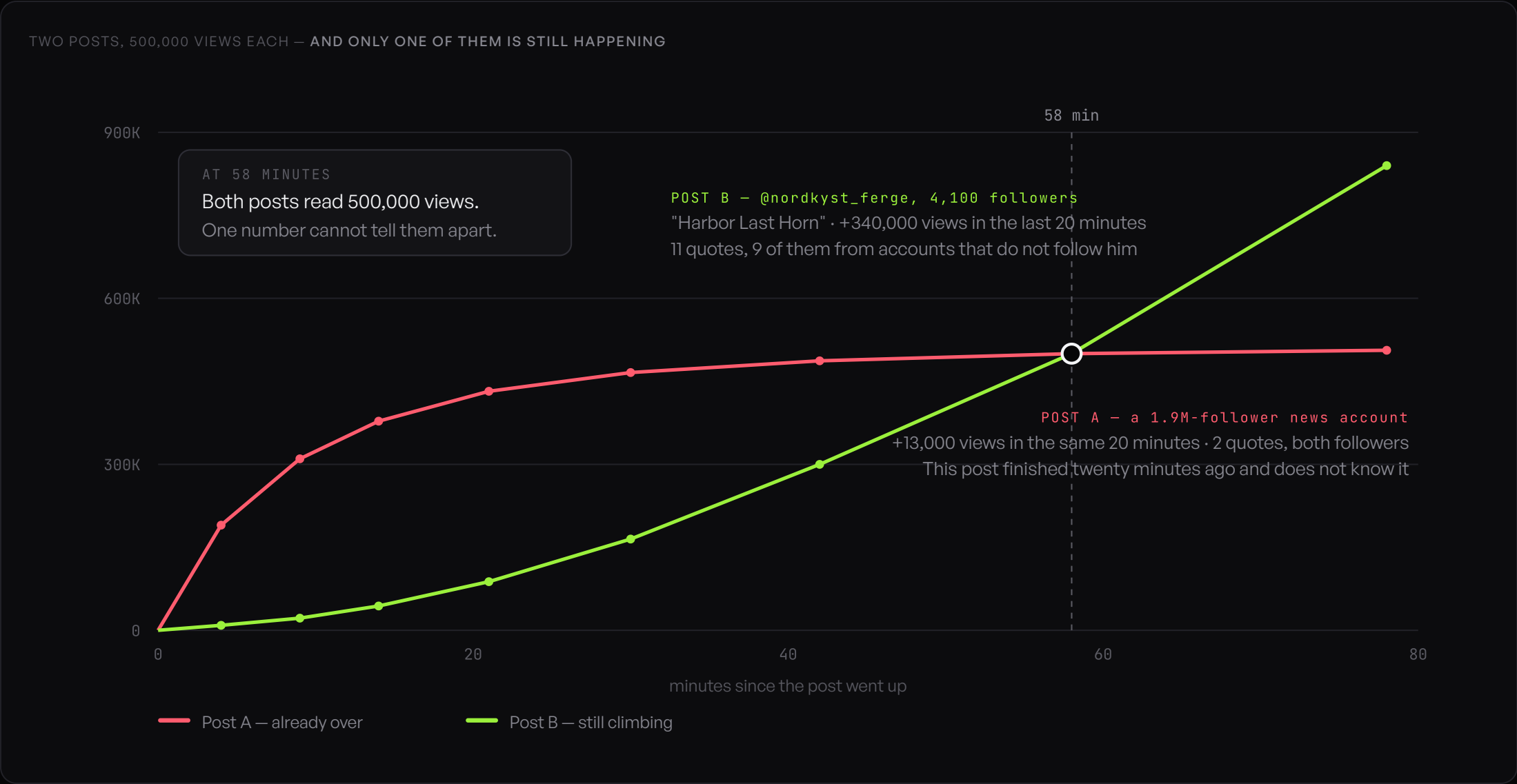
A fixed 2% of everything that arrives runs this full grid regardless of score. That slice is expensive and it is not optional: without it, how long a post's history is depends on how well the post did, and anything we later learn from that history is really just learning "posts we watched a lot did well."

Two readings give a speed, and speed normalises in a way a level cannot: divide by the author's own recent baseline and by what is normal for that platform at that hour, and a tiny account and a huge one land on the same scale. One caution. An author's baseline is under that author's control — push it down with filler posts, then buy engagement on the one that matters. So the per-author comparison only ever re-orders candidates that have already cleared a hard absolute floor. A gate an attacker can open by buying a hundred likes is worse than no gate.

Three readings would tell us whether it is speeding up or slowing down, but three readings is fourteen minutes, and by fourteen minutes the coins have been minted for ten of them and the argument about which one is real is already running without us. So instead we keep two running averages of the same counter — one that reacts in twenty minutes, one over six hours — and compare them. The ratio stands in for the curve, and it exists at the *second* reading. Five minutes earlier, on a product whose whole job is naming the real coin while most of the six-day run is still ahead of it.

Why one reading tells you nothing

This is the chart that justifies the whole apparatus. Two posts. Same platform, same afternoon. At fifty-eight minutes both of them read exactly five hundred thousand views.



Post A's shape is what a big account's own audience looks like: a fast start from people who were already there, then nothing. Post B's shape is what strangers look like: almost invisible for half an hour, then steeper every time we check.

Post A got its five hundred thousand from the 1.9 million people who already follow it, reached that number in fourteen minutes, and has added thirteen thousand views since. Post B belongs to a Norwegian ferry account with four thousand followers. It could not physically have produced five hundred thousand views from its own audience, so nearly all of them

came from strangers — and it is getting faster, not slower. A threshold search would have returned both, and it would have returned Post A first, because Post A crossed the line first.

WHAT THIS BUYS, STATED HONESTLY

This produces candidates, not certainties. It will not beat a bot to a mint — nothing will, and we stopped trying.

What it buys is position. Coins that clear a million dollars take about six days to peak, and **none of them peaked inside the first hour**. So the winnable claim is not "we saw this before the coin existed". It is "of the things minted in the last hour, this is the real narrative, and it is still climbing." Everything on this page exists to make that second sentence true.

The published ceiling on this kind of prediction is a real ceiling: once you control for how early you are — exactly our situation, since every candidate is already fast — accuracy across four independent research domains falls below 65%. We will miss things. The 2% slice we track regardless of score is how we find out *what* we missed, and whether the fault was the chooser or the judge.

07. How posts become stories

One person posting a video of a Norwegian ferry captain is luck. Twenty unrelated people posting their own version of it, on two platforms, inside an hour, is a meme forming. The part of the system that notices the difference is called the Grouper, and it is not a tidying step that bundles duplicates together neatly. It is the detector.

Everything upstream of this operates on one post at a time, and one post at a time cannot tell a hit from an accident. Plenty of single posts get eight million views. Almost none become coins that clear a million dollars. What separates them is whether other people started making their own versions.

The hypothesis this whole product rests on

Memes get copied. News gets consumed. A plane crash gets millions of views and nobody makes their own plane crash — people watch it, react, move on. A ferry captain blowing his horn one last time gets re-cut, screenshotted, drawn and parodied by people who have never heard of each other. A view counter cannot tell those apart; both are a big number going up. Counting how many independent people made their own version tells them apart instantly.

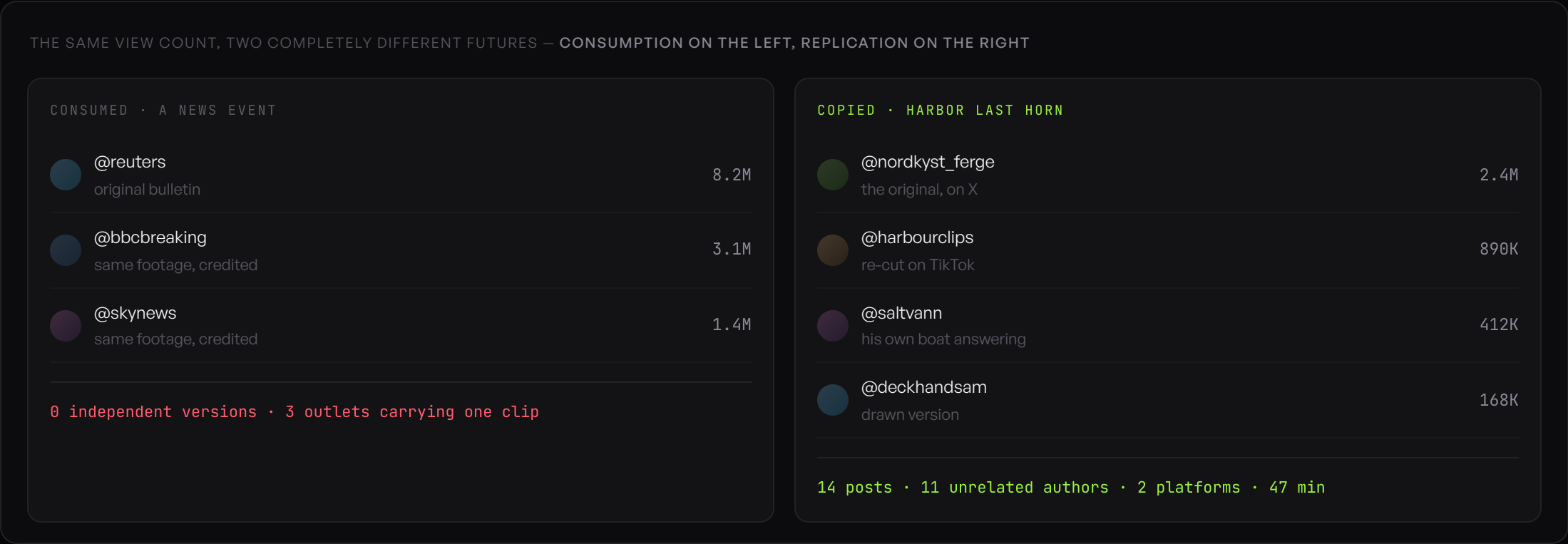


Illustration of the hypothesis, using the canonical example. The left column is a bigger number and a worse signal. Nothing in a view count distinguishes them; the author count does it in one glance.

The four signals that something is being copied

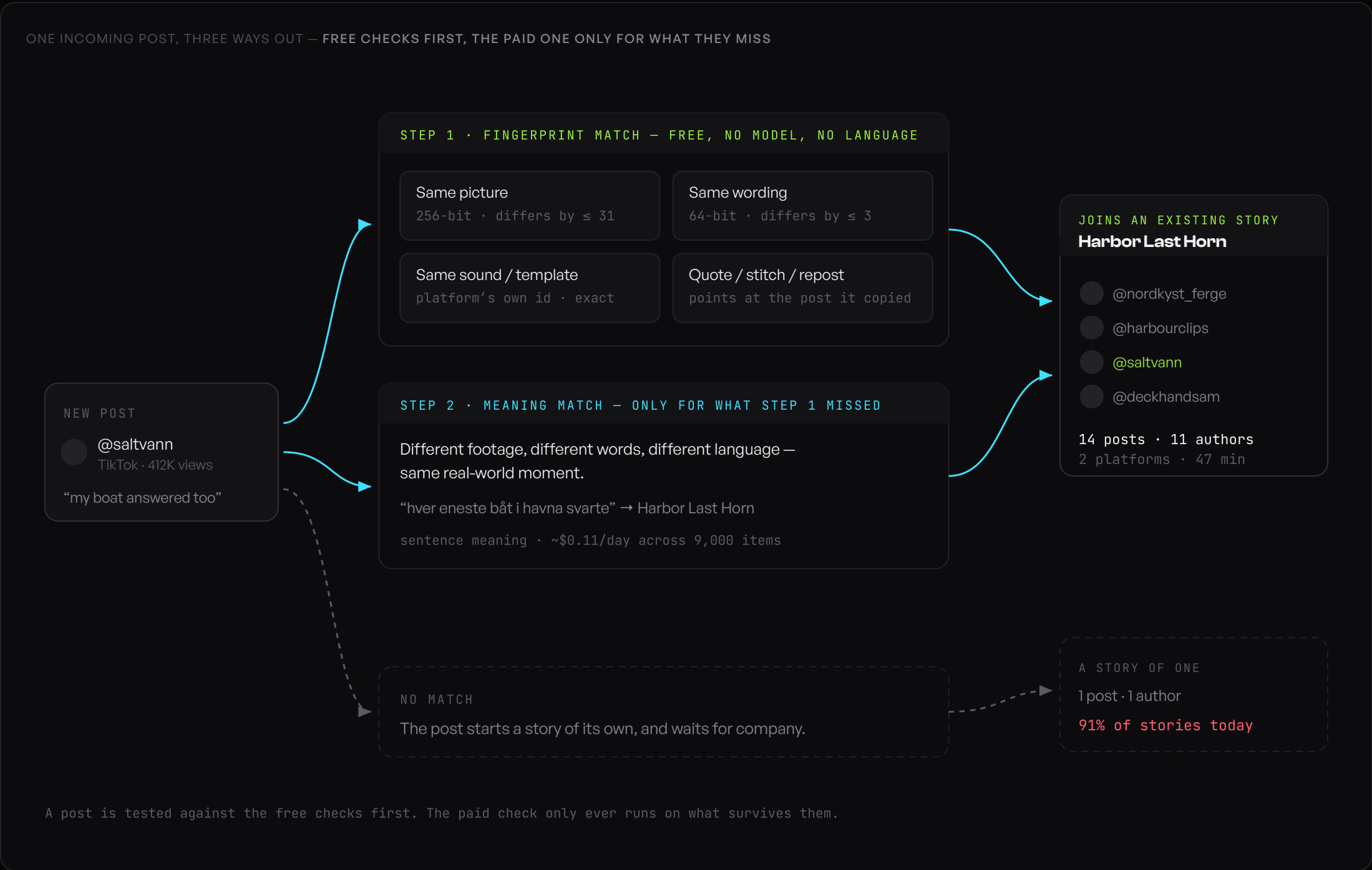
Copying leaves physical traces, and most cost nothing to find. **The same image** reposted by an unrelated account: the fingerprint is identical whether it was posted in Oslo or Ohio, on X or on TikTok. **The crowd naming it in the replies** — "\$FERRY", "that's the one" — which is the crowd telling us which ticker the story has actually attached itself to, and is how the real coin gets told apart from the others minted off the same post. **Variations appearing**: the same joke with new footage, a drawing, another language. And **crossing platforms**, the strongest of the four, because it means the thing escaped the algorithm that launched it.

SIGNAL	WHAT ACTUALLY GETS COMPARED	COST PER CHECK	NEEDS A MODEL?
Same picture	A 256-bit fingerprint of the image, allowing for crops, filters and re-encoding	~2 ms	No
Near-identical wording	A 64-bit fingerprint of the text; catches copy-paste and light edits	~50 μs	No
Same sound or template	The platform's own id for the audio, effect or format	free	No
Explicit pointer	A quote, stitch or crosspost that names the post it came from	free	No
Same meaning, new words	A sentence-meaning comparison — different footage, different language, same moment	\$0.11/day	Yes

The first four are free, deterministic and language-blind. They ship first, and alone. The fifth is the one that needs buying, and it costs about eleven cents a day across nine

thousand items.

How a post joins a story



Two tiers, in cost order. The cheap tier does most of the work and is the reason the Grouper can be multilingual and cross-platform before any model exists at all.

What it does today, stated plainly

THE FAILURE

982 of the 1,079 stories that have any posts in them contain exactly one post, by exactly one author. That is 91%.

The product’s premise is that people are making their own versions. The Grouper, as written, cannot see a second version.

The cause is a single file. What the code calls *embeddings* is not embeddings — it is a word-count map. "Captain", "ferry" and "horn" have to literally appear in both posts for them to be joined. A Norwegian post about the same captain shares no words. A TikTok with no caption shares nothing at all. And "every boat answered him" shares nothing with "the whole harbour blew back", though any human reads them as the same moment.

Worse, there is redundancy nobody had noticed. There are three matching passes, and the third is the second with different weights: any pair the keyword pass accepts, the word-count pass has already accepted. It contributes zero additional matches, at the cost of a third of the code. Meanwhile the word-count pass, with no sense of which words are rare and no penalty for length, invents joins — "Behind-the-scenes of Adwoa Aboah's..." and "Behind the scenes with the champs" score 0.436, over the 0.42 line, and become one story.



The bottom bar is the only one that is a meme forming. It is 2.6% of what the system produces. Everything the feed shows is drawn from a population that is almost entirely single posts wearing the word “story”.

The fix, and the four small things that must ship with it

Real sentence meaning, in place of word counting, is the highest-leverage single change in the entire system. Not a research project — a well-understood bought capability at about eleven cents a day. But dropping it in alone would make things worse, and four small things have to go with it.

- 1. Stop only reading English.** The collector asks X for English posts only. No meaning model can group across languages while the front door is English-only. Moo Deng's earliest signal was a Thai post at 4.6M views, seven to eleven days before the English wave.
- 2. Do not reuse the old cut-off.** The 0.42 line was tuned for word counting. Sentence-meaning scores sit on a different scale, where unrelated sentences routinely clear 0.42. Leaving the number would merge nearly everything into one story.
- 3. Ignore the words that are always there.** One story swallowed 39 unrelated posts because they all mentioned `$SOL`. A list of 45 such words already exists in the codebase, imported by three other modules and by nothing in the grouping path. One import line — the highest value per line of code in the repository.
- 4. Weight words by how new they are, not how rare.** The standard technique punishes any word that suddenly appears everywhere, which is exactly what a brand-new ticker does at the moment it matters. The right measure is how many of the last fourteen days a word has been common: `$SOL` every day, so it counts for nothing; yesterday's ticker counts for almost everything. **That history only accrues forward — if the daily table is not being written today, this feature does not exist in November.**

A RELATED BUG WORTH NAMING HERE

Quotes — someone reposting with their own commentary — is the clearest reproduction signal we have, and the one the product rests on.

The TikTok reader hardcodes `quotes = 0`, because TikTok has no quote feature. So for every TikTok post the core replication signal reads as an honest zero, and nothing objects. Cross-platform copying is currently invisible in one direction.

One cheap experiment should run first: a column already exists that records which pass made each match, and it is empty on every row. Fixing that one write answers which pass earns its keep. It is an afternoon.

Then rules first, a model later. Once the fingerprint tier runs and the meaning tier sits behind a calibrated line, the way to improve it is three hours of hand-labelling — three hundred pairs, "same real-world moment, yes or no" — and fitting a small model to the six match signals in place of hand-set weights. No research, no new service: fingerprints and meaning vectors are both columns in the Postgres database that already runs.

What the user sees when this works

14 POSTS · 2 PLATFORMS

Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

2.4M views on the original

4h old

11 separate authors

X · TikTok

\$HORN \$1.2M

THE POSTS BEHIND IT

	@nordkyst_ferge · X the original · 4h ago	2.4M
	@harbourclips · TikTok re-cut · 3h ago	890K
	@saltvann · TikTok his own boat · 2h ago	412K

Every number on this card would still be true if Insidor did not exist: views, age, author count, platforms, market cap. Nothing about how those posts were joined, how confident we are, or what was filtered out ever appears on screen. “11 separate authors” is a fact about the world. It is also the entire reason the story is here.

08. How stories become coins

A story is not a coin. One meme can spawn hundreds of them, only a few are real, and almost none of them say where they came from. This is the part of the product where being wrong costs a user actual money — so it is built to say "I don't know" more often than it says anything else.

Everything before this was about reading the world. This is the one place we act on that reading. A user sees a story, sees a coin next to it, and presses Buy. If the coin is the wrong one, we did not make a bad recommendation — we spent their money on a stranger's token.

Two measured facts define the problem. First, **a pump.fun search for "chill guy" returns 306 distinct tokens** — one meme, three hundred coins, one or two of which people actually traded. Second, **only about 5% of coins that ever cleared \$50,000 declare the post they came from**. The metadata field exists; it is nearly always empty, and when it is filled it was filled by whoever launched the coin, so it is advertising, not evidence. We cannot ask the coin where it came from. We have to work it out.

What the current build does, and the two coins it bought into

The build that exists today matches a story to a coin by **searching a market data vendor for the ticker as a text string, then picking the result with the most liquidity**. That is the entire method. There is no check that the coin was created after the post. There is no confidence score, and the database column is a plain true/false — a boolean cannot abstain.

TWO REAL MATCHES FROM THE CURRENT BUILD — BOTH WRONG, BOTH ONE CLICK FROM A LIVE BUY

"gym day"
ticker extracted: GYM → PUMP

WRONG COIN

\$PUMP

Pump · minted 375 days before the post

\$1.84B

MARKET CAP

Buy \$PUMP

Marked first_deployed = true.
A \$1.84B token that has existed for over a year, presented as the coin this morning's post created.

"stefan back on the grass"
ticker extracted: GRASS

WRONG COIN

\$GRASS

Grass · minted 633 days before the post

\$225M

MARKET CAP

Buy \$GRASS

Same defect, different token. The string matched. Nothing else was ever checked.

\$PUMP minted

day 0

375 days of causality

"gym day" posted

day 375

The coin existed before the story did. One subtraction rules it out, for free, before any network call — and the current build never performs it.

The gates, cheapest first — and time is the cheapest of all

The fix is not a smarter model. It is putting the checks in the right order, and the right order is by **what each check costs to run**. A subtraction of two timestamps is free. Reading a token's transfer rules is one cheap call. Asking what it would actually cost to buy is a paid, rate-limited call. So we ask them in that order and stop at the first refusal, which means the overwhelming majority of candidates are eliminated without spending anything.

Time goes first for a second reason beyond cost: **it is the only check that is logically decisive**. A coin minted before the post cannot have been created by the post. That is not a probability, it is an impossibility, and it is exactly the check that would have killed both failures above. It also inverts how candidates are found. Today the code searches by ticker and then filters; the rebuild **retrieves by time** — every coin minted in the window that starts at the story's earliest post — and treats the ticker as one scoring signal over that set, never as the way the set is fetched.

GATE	THE QUESTION	WHY IT IS WHERE IT IS	IF IT FAILS
FREE · RUNS LOCALLY · NO NETWORK		ELIMINATES MOST CANDIDATES	
G1	Do we know when this coin was minted?	Mint time is stored with its source and how sure we are. A vendor’s guess can never be recorded as exact.	unknown mint time → out
G2	Was it minted <i>after</i> the post?	The one impossible case. This is the gate that both live failures needed and never had.	predates the post → out
G3	Was it minted soon enough after?	Median post-to-mint lag is 3.8 minutes, and about a minute for the coins that go on to clear \$1M. A coin minted days later is a different event.	too late → out
G4	Is this an established major?	A hard deny-list. No story ever creates a coin that already has a billion-dollar market. Cheap insurance against exactly the \$PUMP shape.	known major → out
ONE CHEAP READ		A HANDFUL OF CANDIDATES LEFT	
G5	Can we read its transfer rules at all?	If we cannot confirm how the token behaves when it moves, we do not offer it. Unreadable is not the same as fine.	unreadable → out
G6	Does it tax or intercept transfers?	Tokens that skim a fee or run custom code on every transfer are a category we simply do not surface.	non-standard → out
ONE PAID, RATE-LIMITED CALL · RUNS LAST		ONLY ON SURVIVORS	
G7	Can we actually get a real buy quote?	This replaces the old "does it have liquidity" test, which was silently deleting almost every new coin — see below.	unquotable → out vendor down → separate reason
G8	Is the all-in cost sane?	Measured all-in cost ranged 1.6% to 22.7% across three coins of the same age. The real number is shown; an absurd one is refused.	otherwise → scored

Read top to bottom, this is also a cost curve. Roughly nothing is spent on the candidates that fail G1-G4, and those are almost all of them.

The liquidity test that deleted our own market

This correction changes what "can this be bought" even means. The old build refused any coin whose liquidity was not greater than zero. That sounds prudent; it is catastrophic. **A brand-new coin on a bonding curve has no liquidity figure at all** — not zero, absent, because that kind of market has no two-sided reserve for the number to describe. Measured live, every bonding-curve pair returned no liquidity object, and the code turned "absent" into "zero" and rejected them. So the filter removed *all the new coins*, which is the entire population this product exists to serve, and kept only the old survivors — a second, quieter reason the matcher kept landing on year-old tokens. The replacement test means what we want: can we get a real quote, at a real price, right now.

The abstain band — why a high score is not enough

Once the survivors are scored on ticker, wording, image and declared links, there is one more rule, and it is the one usually missing from systems like this. **We do not check whether the best candidate scores well. We check whether it beats the second-best by a clear margin.** With 306 tokens named after one meme, a great score on the winner proves nothing if the runner-up scores the same. Ambiguity has to be its own rejection, otherwise "no confident match, no Buy button" is a sentence in a document rather than something the code does.

A story can legitimately have several coins, so each one is adjudicated on its own. The question is never "which single coin is this story" — it is, for each coin we are considering, "is there exactly one convincing candidate, or several that we cannot separate?" Below, one of those contests is decided and one is not.

INTERNAL ONLY — NONE OF THESE NUMBERS EVER APPEAR ON A USER SCREEN

Contest for **\$FERRY** — 6 coins with that ticker minted after the post

CANDIDATE	MINTED AFTER POST	WORDING	IMAGE	DECLARED	TOTAL
"Harbor Last Horn"	+2m 10s	0.88	0.94	yes	0.91
"Ferry Inu"	+3h 12m	0.12	0.05	—	0.38
"FerryCoin 2.0"	+4h 40m	0.09	0.03	—	0.31

Gap of **0.53**. Nothing else is close, so **\$FERRY resolves confidently and its Buy button renders.**

Contest for **\$BOAT** — 31 coins with that ticker minted after the post

CANDIDATE	MINTED AFTER POST	WORDING	IMAGE	DECLARED	TOTAL
"Harbour Boats"	+1h 04m	0.46	0.51	—	0.55
"Every Boat Answered"	+1h 22m	0.52	0.38	—	0.52
"Boat Season"	+2h 51m	0.41	0.30	—	0.44

Gap of **0.03**. Two coins are equally plausible and we cannot tell them apart, so **\$BOAT is listed as unverified with no button** — which is what the user screen below shows. The margin, not the score, is what decides.

CONFIRMED

Clears the gates, beats the runner-up by a margin, and we can quote it.

Buy button renders.

UNSURE

Plausible but contested, or the coin's market is too young for us to be certain.

Listed, no button.

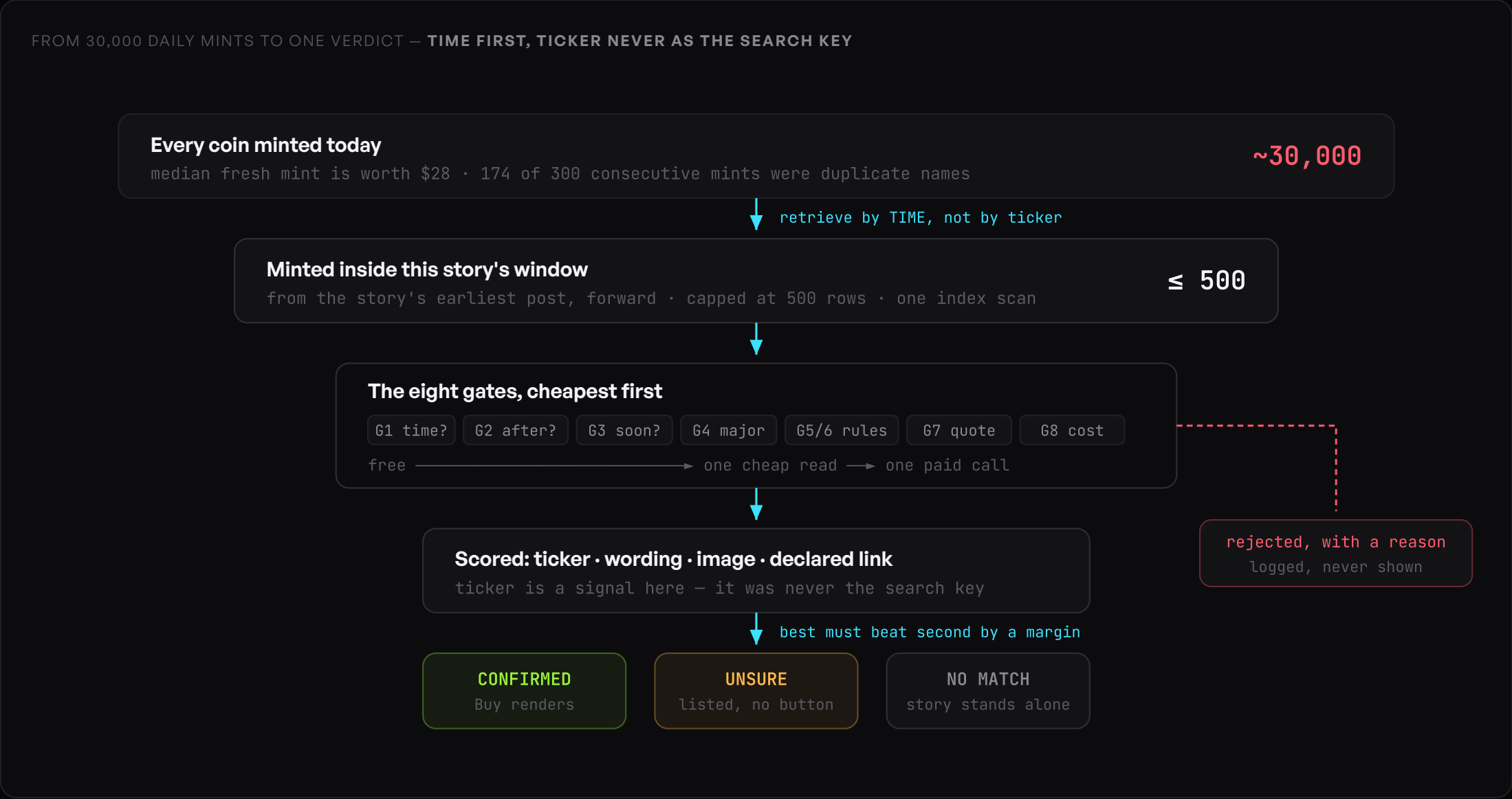
NO MATCH

Nothing survived the gates. The story stands on its own, which is fine.

Nothing rendered.

There is a deliberate consequence baked into this. A venue we have only just started watching can produce **"unsure" at most, never "confirmed"**, until it has enough of its own adjudicated examples — because a score from one marketplace does not mean the same thing on another. A new venue's first month is read-only, and that period is exactly how the examples get collected: every "unsure" is logged with everything the system saw when it hesitated.

The funnel, end to end



What the user actually sees

None of the above reaches the screen. The user gets facts that would be true whether or not Insidor existed: the ticker, what it is worth, when it was minted, and whether it was the first one out or the biggest one now. A coin we are not sure about is still listed — hiding it would be its own kind of lie, since the user can see it on any explorer — but it carries no button.

Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

@nordkyst_ferge on X · 2.4M views · 4h old · also a TikTok clip by @harbourclips, 890K views

\$FERRY minted 4h ago FIRST \$799.6K MARKET CAP Buy
\$HORN minted 3h ago BIGGEST \$1.2M MARKET CAP Buy
\$BOAT minted 2h ago UNVERIFIED \$41.2K MARKET CAP

"First" and "biggest" are observations about the world. "Unverified" is the one word of ours on this screen, and it is a refusal, not a rating.

The test for this whole section: **every wrong answer here is a wrong answer the user paid for**. So the system is tuned to abstain. Expect most stories to carry no confirmed coin, and treat any change that pushes that number up as a change that reintroduces \$PUMP.

09. How the system learns

The idea is smaller than it sounds. Write down what the system saw at the moment it made a call. Days later, write down what actually happened. Put the two lists side by side and you have taught it something. That is the whole machine-learning plan, and it starts working before there is any model at all.

Right now the system makes a few thousand calls a day — this post is worth watching, that one is not, these ten posts are the same story — and remembers none of them. Tomorrow it makes a few thousand more, no wiser than yesterday. The reason is not that it lacks a model. It is that it never wrote anything down. A model built on nothing is still nothing.

The awkward gap: minutes to decide, days to find out

The decision has to happen fast. A post goes up and a coin appears a median of **3.8 minutes** later. Whether that coin was worth anything takes about **six days** to know — that is when the typical winner peaks. Every call is made on minute-old evidence and graded on week-old evidence. In that week the post keeps moving and the replies fill up, so anything not captured at the moment of the decision is not merely harder to get later. It is a different thing wearing the same name.

Two lists, one join

So: two tables. The first is written the instant a call is made and never touched again. The second is written days later by a job that goes and checks. They share one key.

THE TWO LISTS — FROZEN AT THE MOMENT OF THE CALL, JOINED A WEEK LATER

What we saw

WRITTEN 08 AUG · 16:04:11

SUBJECT	story_7f3a · Harbor Last Horn
STAGE	admit
POST AGE	4h 02m
VIEWS	2,417,880
REPOSTS / QUOTES	18,402 / 6,115
RETELLINGS	11 accounts, 3 platforms
NEWEST INPUT	16:03:58 — 13s old
CALL	pass
JUDGED AGAINST	policy 4c1e·9a
WHO DECIDED	rule:admit@3
HELD-BACK SLOT	no

What happened

WRITTEN 14 AUG · 04:00

SUBJECT	story_7f3a · Harbor Last Horn
MEASURING	best coin peak / first price
WINDOW	6 days from the post
POPULATION	all admitted stories
COINS FOUND	\$FERRY, \$HORN, \$BOAT
FIRST COIN MINTED	12:09 — 7m after the post
BEST PEAK	\$1,204,900 (\$HORN)
WHERE IT SITS NOW	\$14,780
STATE	resolved
VERDICT	yes — it was worth surfacing
MEASURED BY	peak_multiple v1

JOINED ON

story_7f3a — the same key on both sides. One line of the left list plus one line of the right list is one row of training data. Repeat a few thousand times a day and after three months you have a dataset nobody else on earth has.

Left is written in the same millisecond as the call and is never edited. Right is written by a job that wakes up a week later and goes and looks. Neither list knows the other exists until they are joined.

Notice the line that says *who decided*. Today it reads `rule:admit@3` — a threshold somebody typed. In six months it might read the name of a trained model instead. **The row does not care.** It records what was seen, what was decided, and what bar it was decided against, and the outcome half is identical either way. That is why this works from day one with no model in the building: every call the hand-typed thresholds make today becomes a labelled example for the model that replaces them. The current build has no machine learning in it at all. This is how it gets some, without a single line of it existing yet.

Why this cannot be bolted on later

The obvious objection is that we already store posts and view counts, so surely the training data could be reconstructed later from what we have. It cannot, for two reasons, and both are already true of the code running today.

TWO REASONS THE PAST CANNOT BE RECONSTRUCTED — BOTH MEASURED IN THE CURRENT BUILD

1 · THE SAME QUESTION, ASKED TWICE, GIVES TWO ANSWERS

2 · HOW LONG A HISTORY IS DEPENDS ON HOW IT DID

Readings kept, per post

A post that ran

84

A post that stalled

3

Slow posts are dropped after ten minutes; everything stops being watched at three hours. So the length of the record is a clue to the answer.

Left: the speed field has no "as of" filter, so asking for it later returns a different quantity under the same name. Right: watching stops at three hours and quiet posts are pruned at ten minutes, so winners carry long records and losers carry stubs.

WHAT TRAINING ON A RECONSTRUCTED PAST ACTUALLY TEACHES

"Posts with a lot of stored readings do well."

Which is perfectly true, completely useless, and impossible to spot from inside the numbers — the model would score beautifully in testing and be worthless in the morning. This is not hypothetical: it is the same shape of mistake that voided the backtest, where the strongest-looking signals were read out of reply threads months after the coin had already run. A coin that goes up gets talked about. "People were talking about it" then looks like a brilliant predictor of the thing that caused it.

Four states, and everybody gets the third one wrong

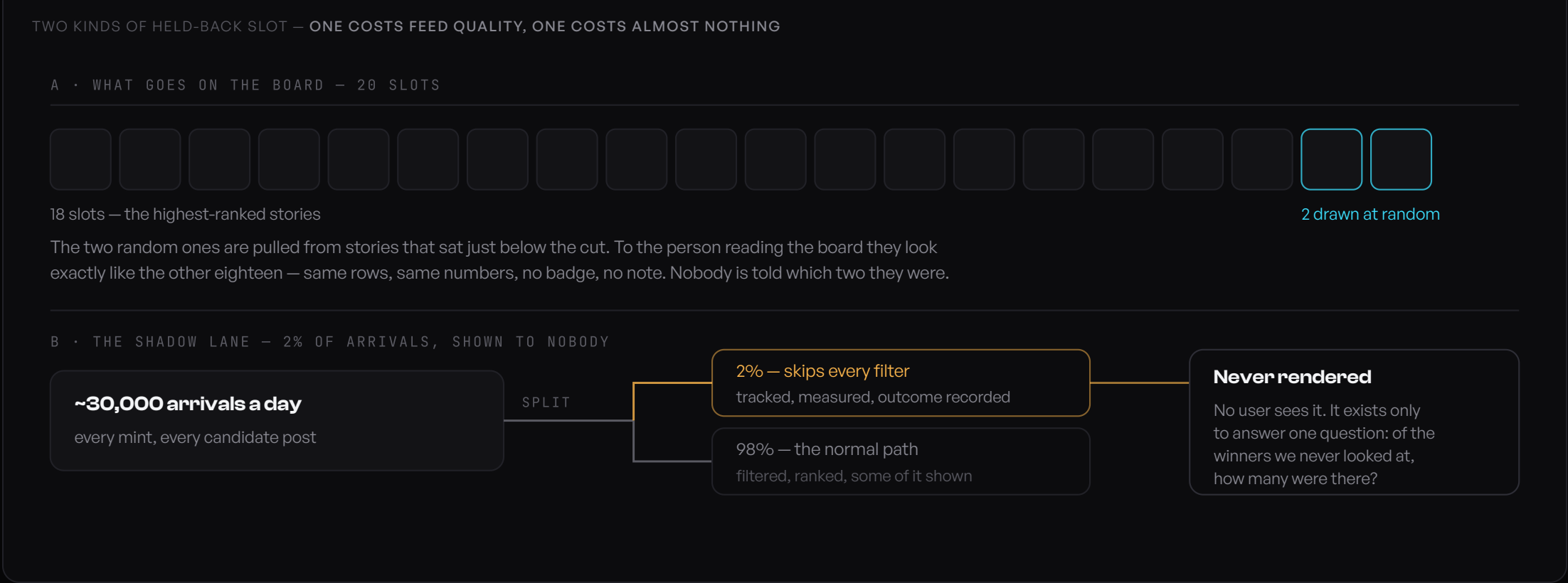
The outcome half has to admit when it does not know yet. With six days to a typical peak, a large share of rows are simply unfinished at any moment, and quietly filing those as failures would poison everything downstream.

STATE	MEANS	WHAT IT LOOKS LIKE	USED FOR TRAINING?
Yes	peak \$1.2M	The window closed and the thing cleared the bar.	yes
No	peak \$2.8K	The window closed, nothing happened, and we were watching the whole time. That last clause is the hard part and it needs its own record.	yes
Not yet	day 2 of 6	Still inside the window. Never counted as a no. This is the classic error and here it would be severe.	no — held back
Can't say	gap in the feed	The mint feed dropped out, or the post was deleted, or the counter went stale.	no — but counted, because a rising count is the first sign the pipeline is rotting

The fourth row earns its place by being an alarm. If the share of unanswerable rows climbs, something upstream broke silently — which is exactly how the TikTok field mix-up went unnoticed, quietly reporting a real signal as an honest zero.

Holding a few slots back on purpose

There is a trap in a system that only ever acts on its own best guess: it only ever finds out about the things it already liked. Everything it declined to look at stays permanently unknown, so the record slowly becomes a portrait of its own habits. The fix is to give away a small, deliberate, budgeted amount of the decision to chance.



The two random board slots cost feed quality and buy honest evidence about the boundary.
The shadow lane costs about two per cent of the watching budget, costs zero feed quality,
and is the only thing that ever measures what the filters threw away.

The shadow lane is the one almost nobody builds, and it answers a question no other measurement can. Suppose we are missing winners. Is that because the ranking is bad, or because the front gate never let those posts through at all? From inside the normal path the two are indistinguishable, since everything the gate refused is simply absent from the record. Compare the shadow lane's hit rate against the gated one and the answer falls out: if the shadow lane is full of winners the gated path never saw, the *gate* is the problem, and no amount of model work will fix it.

The two random board slots have a price, and it should be written down with a number attached or it gets quietly deleted the first bad week. On a twenty-row board it costs roughly **0.6 winners a day not surfaced**. That is the fee for being able to discover something unlike anything the system has seen before, and somebody should own it. If money ever forces a cut, cut the random slots and keep the shadow lane: random slots can be switched back on, but a missing shadow lane leaves a permanent hole no future work can fill.

One related rule follows from the same worry and is enforceable in code: **nothing may score a story by how much it resembles a past winner**. That is the fastest route to a system that finds only derivatives, which is worthless in a market whose entire product is the thing nobody predicted. One number polices it forever — the hit rate on the winners *least* like anything seen before. If that collapses, the system has become a copy machine no matter how good the headline looks.

The loop, end to end



Steps 3 and 6 are the two that do not exist today. Everything else in the loop is either already running or is a week of work. The gap between them — the six days — is why they cannot be written retrospectively.

THE ONE LINE THAT MATTERS FOR SCHEDULING

Every day this is not recording is a day of training data that can never be bought, rebuilt, or backfilled.

The rest of the build can slip and cost time. This one slips and costs the asset — which is why week 3 is drawn as a point of no regret rather than a milestone. It is also the least glamorous work in the plan: it ships nothing a user can see, and it is the only part that makes the company harder to copy in a year than it is today.

One last thing, because it is the rule the whole deck exists to enforce: none of this ever appears on a screen. The two lists live in a part of the database the public app is structurally incapable of reading — not "we remembered not to expose it", but a separate compartment the public interface has never been told about. That matters more here than anywhere, because these tables hold every threshold, every held-back slot and every number the system looked at, which together is a complete recipe for gaming the gates. The user sees a story, a view count, an age, and a coin. What we thought about it stays inside.

10. What we store, and what it costs

Eight kinds of thing get written down — six that make the product work and two new ones that let it improve — and the whole machine runs on about six pounds a day. This is the honest accounting: what sits in the database, why each piece has to be there, and where every pound goes.

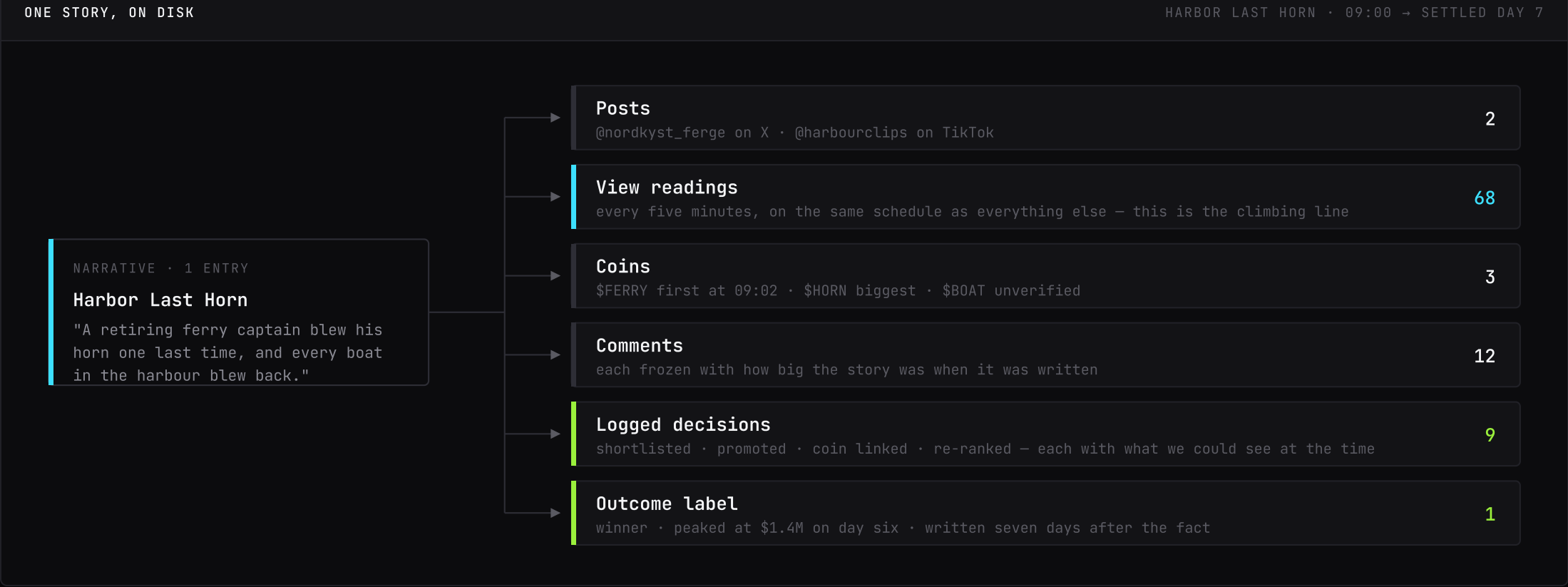
The database is one central store and the single source of truth. Everything else reads from it. There are six kinds of record that the product itself needs. **Narratives** are the stories: one entry per viral moment, holding the plain-English two-liner and the pointers to everything else. **Posts** are the individual social posts that make up a story — the original X post, the TikTok clip someone else put up twelve minutes later. **View readings** are the important one: every time we re-check a post, we write down the clock time and the view count. That is what draws the climbing line, and it is the one thing we could never buy back later. Nobody sells "what did this post look like twenty minutes ago". If we are not recording it as it happens, it is gone.

Coins are every coin we have tied to a story, with a note of how sure we are about the link. This is a bigger job than it sounds: a search for one meme returned 306 separate tokens, so most of the work is deciding which one is real. **Comments** are the traders' theses, each stamped with how big the story was at the moment it was written and frozen permanently, so a call made early can never be quietly reframed as a call made late. **Running costs** are the day's spending on data and AI, written down as we go, so the system knows what it has spent and can stop itself.

Two more are new, and they exist for one reason: the current build makes a few thousand decisions a day and remembers none of them. The **decision log** records every judgement at the moment it is made — what we were looking at, what we could see at that instant, what we decided, and why. The **outcome labels** record what actually happened afterwards, settled a week later when the answer is known. Put those two together and you have the only thing that can ever make this system smarter: a list of guesses with the answers attached.

THE EIGHT THINGS WE WRITE DOWN		VOLUMES ARE A NORMAL DAY AT LAUNCH SCALE	
WHAT WE STORE	ONE ENTRY IS	FOR "HARBOR LAST HORN"	NEW A DAY
Narratives THE STORY	One viral story, with the two-liner a human reads first.	"A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back."	30 ENTRIES
Posts THE SOURCES	One social post belonging to a story — who posted it, where, and when.	@nordkyst_ferge · X · 09:00 @harbourclips · TikTok · 09:12	1,200 ENTRIES
View readings THE HISTORY	One re-check of one post: the time, and the view count at that time. Never overwritten, never pruned.	09:04 → 41,000 09:09 → 118,000 09:14 → 264,000 ...	44,000 READINGS
Coins THE MARKET	One coin considered for a story, and whether we linked it. 30,000 coins are minted a day; almost all are noise.	\$FERRY · first · 09:02 \$HORN · biggest \$BOAT · unverified	30,000 SEEN · 50 LINKED
Comments THE THESES	One trader's written call, with the story's size at that moment frozen in.	12 comments earliest stamped "\$61K · still climbing"	140 COMMENTS
Running costs THE METER	What we have spent today, per supplier, updated as each request is made.	not per story — one line per supplier per day	2 LINES
Decision log THE GUESSES NEW	One judgement, at the instant it was made: what we could see then, what we chose, and why. Append-only — nothing is ever edited.	09:06 shortlisted 09:11 promoted to feed 09:33 real coin identified · first	4,200 DECISIONS
Outcome labels THE ANSWERS NEW	What actually happened, settled seven days later. Winners reached about 18× their opening size; losers, 1.19×.	\$FERRY · winner peak \$1.4M on day 6 settled 16 Aug	30 SETTLED
Three schemas, one database: what the app serves · what we record to learn from · raw captures kept for re-reading. Same machine, separate rooms, so a change to one cannot quietly reshape another.			

View readings dwarf everything else — forty-four thousand a day against thirty stories. That is correct. The readings are the asset; the story is just the label on them.



Sixty-eight readings, nine decisions and one answer, for one story. Delete the readings and you still know the ferry captain went viral — you just can no longer tell whether he was climbing at 09:09. Delete the decisions and you can never find out whether we were right.

What was broken about the old way of writing things down

The build that exists today cannot learn from itself, and not for want of data — for want of four specific habits. Three of them quietly corrupt the record; the fourth corrupts a number the product depends on. All four are storage problems, which is why they belong in this section rather than a later one.

FOUR WAYS THE RECORD WAS SPOILED		ALL FOUR ARE FIXED BY HOW WE WRITE, NOT BY WHAT WE BUY
THE HABIT	WHAT IT DID TO THE RECORD	WHAT WE DO INSTEAD
Decisions were never written down A FEW THOUSAND A DAY	The system judged constantly and kept none of it. There is no way to ask "what did we think of this story at 09:06", because nobody wrote it down.	Every judgement is logged as it happens, append-only, with the view of the world it was made from attached.
Numbers had no "as of" time THE GROWTH FIGURE	Asking for a post's growth rate later returned a <i>different answer</i> under the same name. The number moved; the question did not.	Every stored number carries the moment it was true, and asking about 09:06 always returns the 09:06 answer.
The record length gave away the answer SURVIVORSHIP	Re-checking stopped three hours in, and slow posts were dropped after ten minutes. Winners ended up with long histories and losers with short ones — so any model trained on it learns "posts with lots of readings do well". True, and useless.	Same schedule for every post, winner or dud, for the same length of time. A boring history is data too.
Fields were filled with the wrong thing THE TIKTOK BUG	TikTok plays were written into a column named <code>views</code> , shares into one named <code>retweets</code> , and quotes were hardcoded to <code>0</code> because TikTok has no such thing. Quotes are the signal the product rests on — so every TikTok post reported an honest, invented zero.	Each platform's numbers keep their own names, and a thing that platform does not have is stored as blank, never as zero.
Why nothing objected: the old code had no way to describe the shape of a record, so nothing could be checked against it. The new stack does — which is why three of these four become impossible rather than merely discouraged.		

The third row is the expensive one. It is the kind of mistake that produces a confident, well-behaved model that has learned nothing except how long we kept watching.

Five streams, and the week that decides them

Five things about the world are perishable: they exist for a moment and then stop being observable at any price. Today only one of the five is being recorded properly. Three are not being recorded at all, and a fourth deletes itself every twelve hours. Everything else in the plan can slip and only cost us time. These cannot: a week not recorded is a week that can never be bought, borrowed or reconstructed.

THE FIVE PERISHABLE STREAMS

STATUS TODAY · ALL FIVE RECORDING BY WEEK 3

01

Post view readings

how a post's attention climbed, minute by minute

PARTIAL · WRONG SCHEDULE

fixed in week 1

02

The mint stream

every coin created, in order, with its name and its first minute

NOT RECORDING

starts week 2

03

Coin size over time

market cap and liquidity from mint to peak to collapse

NOT RECORDING

starts week 2

04

Our own decisions

what we judged, when, and from what evidence

NOT RECORDING

starts week 3

05

Raw post captures

the untouched original, for re-reading when we change our minds

DELETES ITSELF EVERY 12H

kept from week 3

Week 3 is the point of no regret.

After it, a slip costs time. Before it, a slip costs the asset – and the asset is the only part of this that a competitor cannot simply buy.

Nothing on this list is difficult. Each is a small job that has to start early, because the thing it records is only available while it is happening.

The scores we keep, and never show

We also calculate internal scores. How fast a post is growing. How memeable the AI judged it. Whether a story has cleared the bar to appear at all. These are stored alongside everything else, they decide which stories reach the feed and in what order, and they are the reason the product works. They are *never* shown to a user and never sent to their browser. The app's job is to make the judgement so the user does not have to; publishing our workings would hand them the job back. A trader opening the app has about ninety seconds. Give them a heat score of 8.4 and you have not helped them decide — you have handed them a second decision on top of the first.

STORED, LOGGED, USED TO DECIDE – STAYS ON OUR SERVERS NEVER LEAVES THE MACHINE · NEVER IN THE APP	
How memeable the AI judged it	meme 7
Growth between two readings	+18.4% / 20m
Acceleration	3.1x
The combined ranking score	heat 8.4
Gates and thresholds	author cap 2
How sure we are a coin matches	match 87%
Position in the ranking, and its change	rank ▲4
Which AI model made the judgement	model name
What we looked at and dropped	40 of 63
The outcome label, once settled	winner 18x
SENT TO THE BROWSER – FACTS ABOUT THE WORLD TRUE WHETHER OR NOT INSIDOR EXISTS	
The two-liner, in plain English	the sentence
How many people have seen the post	2.4M views
How old it is	4h · posted 09:00
Who posted it, and where	@nordkyst_ferge · X
Whether attention is still climbing	a rising line
Market cap of the coin	\$799.6K
Liquidity in the pool	\$84.2K
How many people hold it	1,204 holders
Which coin came first, and when	\$FERRY · 09:02
What other traders wrote, and when	12 comments
The test: if a number would still be true with Insidor switched off, it can go on screen. If it only exists because we calculated it, it does not. The struck-through values on the left are printed here so the team can recognise them on sight – they never appear in the product.	

The left column is the company. The right column is the product. Nothing crosses from left to right, in any screen, at any time.

BEFORE YOU PUT A NUMBER ON SCREEN, ASK	
Would this number still be true if Insidor did not exist?	
YES → IT CAN BE SHOWN	
2.4M views – the post has them either way	
\$799.6K – on-chain, public, checkable	
4h old – the clock is not ours	
1,204 holders – anyone can count them	
NO → IT STAYS BEHIND THE SCENES	
meme 7 – we invented the scale	
accel 3.1x – needs our own history to exist	
match 87% – our uncertainty, not theirs	
rank ▲4 – a fact about our list, not the world	

One question, applied to every field, settles the argument in five seconds. It has a useful side effect: the numbers that pass are exactly the ones a sceptical user can verify elsewhere, so showing them builds credibility. The numbers that fail are the ones we could never prove anyway.

What it costs

About **six pounds a day, roughly £181 a month**. Reading social posts is by far the biggest line and the only one that scales with how hard we look — two pounds in every three. The AI judging is a little over a pound. The infrastructure — one database, the loops, a watchdog on separate hardware, and the app itself — comes to under a pound a day combined, and the app hosting is genuinely free at this size.

WHERE EVERY POUND GOES				PER DAY · STEADY STATE · LAUNCH SCALE	
READING SOCIAL POSTS · £4.00		AI · £1.20	INFRA · £0.84		
66%		20%	14%		
LINE	SHARE OF THE DAILY SPEND	PER DAY	PER MONTH		
Reading social posts the data reseller — rented, metered and capped		£4.00	£120		
AI judging Claude, reading the fastest-growing posts and writing the two-liner		£1.20	£36		
Database one Postgres, three schemas — holds all eight kinds of record		£0.50	£15		
The loops the collecting and measuring, running around the clock		£0.27	£8		
The watchdog separate hardware, so it can still shout when everything else is down		£0.07	£2		
The app the feed and the shareable story pages	FREE TIER	£0.00	£0		
Training runs offline, occasionally, on a machine we already own	NO CASH COST	£0.00	£0		
Total		£6.04	£181		
Storage, for scale: a year of readings, decisions and labels is roughly four gigabytes — about fifty pence a month of disk. The irreplaceable part of this business is the cheapest line on the page.					

Recording everything is not what makes this expensive. Looking is. Storage is close to free, which is worth saying plainly: the argument for keeping the decision log and the outcome labels never has to be won on cost, only on discipline.

The spending limits are hard, not advisory. Before every request to the data reseller or to the AI, the system checks what it has already spent today against the cap. If the cap is reached, collection **pauses until the next day** — the machine stops rather than quietly running up a bill overnight while nobody is watching. A runaway loop or a bad day of traffic costs us the rest of that day's stories, which is annoying. It does not cost us a four-figure invoice, which is not.

THE DAILY CAP	OUR OWN METER — NOT A SCREEN ANY USER SEES
SPENT TODAY · READING SOCIAL POSTS £3.42 / £4.50 76% of today's allowance at 18:40. Normal. The measuring keeps running on its usual five-minute cycle, and every reading is still written down.	
SPENT TODAY · AI JUDGING £1.18 / £1.50 Above 75%, so only the fastest-growing posts are sent to the AI for the rest of the day. At 100% the requests stop entirely and resume at midnight — and the fact that we stopped is itself written to the decision log, so a thin day is never mistaken later for a quiet one.	
Ceiling: £6.00 a day across the two metered suppliers · infrastructure is fixed at about 84p · so the worst possible day is £6.84 . Reaching the ceiling pauses collection; it never authorises an overspend.	

PART THREE

How we build it

The languages, the structure, the order the pieces get made in, and how long it honestly takes.

-
- 11 The stack, and why
 - 12 Where the code lives
 - 13 How each part gets built
 - 14 The timeline, honestly

11. The stack, and why

Part three is about how this gets built. It starts with the language, because one live bug settles that argument better than any amount of principle: today the product's core signal reads as an honest zero for every TikTok post, and nothing in the codebase was capable of noticing.

The bug that answers the language question

The usual case against JavaScript is "it lets you make mistakes." True, and not good enough. The current build is 164 files and 25,776 lines of working JavaScript that does real work every day. Nobody is struggling to write it.

The real problem is narrower and much worse. This system is built on promises about *shape* — an adapter must hand core our vocabulary, not the vendor's — and in plain JavaScript there is no way to make a shape mandatory. It can only be described in a comment. Here is what that costs, in the file that reads TikTok posts.

One TikTok post, translated

the harbour clip by @harbourclips, as it exists today

worker/ingest/lib/parse-tiktok-post.js:82-86

WHAT TIKTOK SENDS their field names		WHAT WE WRITE IT INTO our column names, today		WHAT THE PRODUCT THEN BELIEVES downstream, with no idea it's TikTok
playCount	890,400	→	views	A play is an impression
diggCount	148,900	→	likes	Close enough
commentCount	4,192	→	replies	Fine
shareCount	12,438	→	retweets	A share is a repost
collectCount	9,517	→	bookmarks	Fine
— nothing	no such thing	→	quotes = 0	“Nobody has copied this.” THE SIGNAL WE SELL

TikTok has no quote-post. The parser hardcodes **quotes = 0** because there is nothing to put there. Downstream, **quotes is the reproduction count** — other people making their own version — and it is the single strongest thing this product looks for. Every TikTok post therefore arrives looking uncopied. Nothing objected, because there was no shape to object to.

The 890K views on the harbour clip are real; so are the shares. The zero is not a wrong number, it is a category error wearing a number's clothes — and a category error is invisible to a reviewer scanning a file, because every line in it looks correct on its own.

What types actually buy

A contract is a plug shape. Right now the system has plug shapes drawn on paper next to each socket, and everyone agrees they should match. Occasionally someone plugs in the wrong thing, it fits physically, and the appliance runs slightly wrong for six months. Types make the plug shapes physical. The wrong adapter does not fit. You find out while you are holding it, not in a report three weeks later.

Under the new shape, an adapter has to declare what its platform *cannot* count. TikTok’s adapter says "reproduction: absent" — not zero, absent — and core switches to deriving copies from the fingerprint index instead. You cannot ship an adapter that quietly claims a counter it does not have, because the declaration is required to build the object at all. The same move fixes two other promises this architecture is resting on:

Three promises the design makes all three are currently sentences in a document		
THE PROMISE	IN PLAIN JAVASCRIPT, TODAY	WITH A SHAPE THAT IS CHECKED
An adapter returns our shape, not theirs <code>adapters/platform/tiktok</code>	A missing counter becomes 0 and travels the whole system as a fact. The harbour clip looks uncopied.	The adapter must list what the platform lacks. Omitting it does not compile.
A censored reading emits no rate, never a zero <code>core/kinetics/rate</code>	A rounded counter that hasn't visibly moved publishes 0 per minute , which reads as cooling, which demotes the item that is actually accelerating.	"No usable reading" and "a reading of zero" are different shapes. Reading a rate off the censored one is an error in the editor.
A decider cannot reach the network <code>core/*/stage.ts</code>	Nothing stops a stage fetching fresher data mid-decision, which silently breaks replay. The import rules already written down enforce nothing — the plugin was never installed.	A function declared to answer immediately cannot wait for anything , so it cannot fetch, query, or call a model. The guarantee is the signature.

Replay — "re-run every past decision under a new rule and see what changes" — is the reason the whole system is worth building. It only holds if deciding is pure. In JavaScript that is a promise; in TypeScript it is a shape the compiler will not let you break.

The decisions

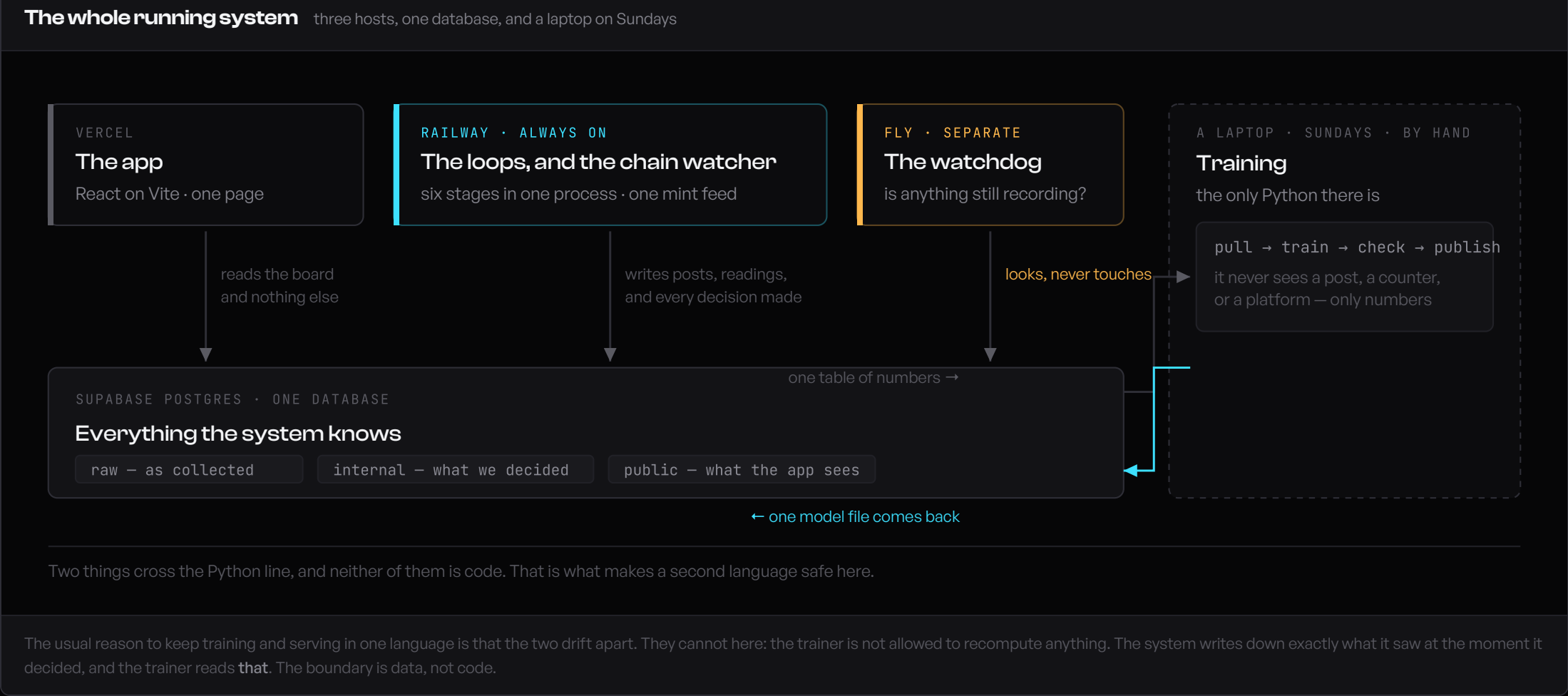
Every one of these is made, not surveyed. Three of them are irreversible in the sense the architecture section means; the rest are files, and you will change two or three of them over a year at a cost of days.

Ten choices, each with its exit performance played no part in any of them — the workload is 0.35 events a second			
SLOT	THE CHOICE	WHY, IN ONE LINE	WHAT WOULD CHANGE IT
LANGUAGE	TypeScript everywhere, except one offline folder LOCKS IN	The design is made of contracts, and a contract nothing checks is documentation. See the zero above.	Nothing at this size. Past roughly six engineers, Go for the services becomes arguable.
RUNTIME	Node 24 , long-term support; Node 22 stays as the rollback	It runs typed files directly with no build step, is supported to 2028, and every vendor kit we are forced to use is Node-shaped.	A dependency with no Node 24 build — drop to 22, alive until April 2027.
PACKAGES	pnpm , one workspace of nine folders	The only one that makes "adapters cannot reach into core" true at install time rather than at review time. Measured: the npm equivalent does not enforce it.	Nothing. It is the cheapest structural guarantee on offer.
BACKEND	No framework. Six supervised loops in long-lived processes	The work is six timers and a third of an event per second. A web framework here is ceremony with a maintenance bill.	If work ever became user-triggered and bursty instead of scheduled.
FRONTEND	React on Vite , one page app — not Next.js	Nothing here is search-indexable or cacheable, and every number is stale within a tick. Server-rendered HTML would be wrong before it reached the screen.	If the read API had to become private and signed-in.
DATABASE	One Postgres , three schemas, vector search built in	The similarity search we need is already in it. A second store is a second backup story, a second outage, and a second thing to restore — for two people.	Nothing at 55M decision rows a year. Revisit at fifty times that.
TRAINING	Python , one folder, run by hand on a laptop, weekly	Three seconds of a standard library beats writing our own. It is never running when the system is running.	Nothing.
SERVING	TypeScript ; the meaning-vectors come from a paid API	Running a trained tree is arithmetic, about sixty lines. Hosting our own encoder costs about \$7 a month of memory to avoid a \$0.33 a month bill.	Volume up a hundredfold, or vectors needing to be reproducible forever.
HOSTING	Railway for the loops · Fly for the watchdog · Vercel for the app	The watchdog must not sit on the same control plane as the thing it is watching, or one provider outage takes the alarm out with the house.	If Vercel's preview builds go unused, the app moves to Railway too.
ROLLOUT	One stage at a time , both builds sharing one database LOCKS IN	Collection cannot stop while we work, and a single switch-over of six coupled stages has no way back.	Nothing. Build-fresh-and-switch is ruled out by team size, not by taste.
Two rows carry a locks in mark: the language and the rollout shape are decided once. Everything else on this page is a file you can change in an afternoon and a deploy you can undo.			

The row worth arguing about is the frontend. Next.js is the default answer in 2026 and it is the wrong one here for a specific reason, not a stylistic one: its whole advantage is sending finished HTML, and finished HTML about a market is out of date by the time it arrives.

What runs where

Four boxes and one database. The important detail is the dotted one: Python is in the repository and never in production. It reads a table of numbers the system already wrote down at decision time, and hands back a model file. If Python vanished from the laptop at three in the morning, the system would keep making decisions with the file it already has.



The watchdog sits on a different provider on purpose. Its entire job is to say "nothing has been recorded for eleven minutes" — and it cannot say that if it goes down with the thing it is watching.

What this costs, honestly

Four real costs, none of them "learn a new language." They are listed because the earlier research got the first one wrong in the optimistic direction, and a plan built on the optimistic version would have hit all four in week one.

01

The build step is gone, the checker is not

Node runs typed files directly — but it strips the types without checking them. You still install the checker and still run it in the background.

The upside is real: **the developer is never blocked by it**. He runs the file; the check reports later.

02

Four features crash at 3am instead of in the editor

A handful of older TypeScript features cannot be stripped, so using one fails when the code loads, not when it is written. In a system touching money that is the worst failure shape there is.

One line of config turns all four into editor errors. Not optional; goes in the first commit.

03

Import shortcuts do not work

The tidy path aliases most projects use are invisible to Node here. Imports have to be written a specific, slightly uglier way.

A layout decision forced by the runtime — so it belongs in the **first commit, not the fortieth**.

04

Two weeks of friction, then it stops

Strict settings generate hundreds of complaints on day one. Bounded and fixable by staging them.

The two noisiest stay **off until the system runs**. Types are mandatory only at the edges — ordinary function bodies are left alone.

One thing types explicitly do not buy: they vanish at runtime, so they prove nothing about the JSON a vendor actually sends. That is a separate, unavoidable piece of work — a checker per platform, run against recorded real responses, so a vendor changing shape fails the build. That cost is identical in every language, and believing otherwise is the most common way a team thinks it is protected when it is not.

The short version: the build is not wrong because JavaScript is sloppy. It is wrong because this design is made almost entirely of promises about shape, and the current language cannot hold one. The zero on the harbour clip is what that looks like in the product — not a crash, not an alert, just a quietly wrong answer to the one question the whole thing is asking.

12. Where the code lives

You told us you could not find the logic in the build we have. That is not a documentation problem, and no README fixes it. The folder names have to answer the question on their own — so the shape below is designed around one measurable test: how long it takes a person who has never seen this repository to open the right file.

The current build fails that test for a specific reason. The place where a post is judged worth coining is `worker/adapters/anthropic/meme-score.js` — a file inside a folder named after a vendor. It reaches back into `worker/score/lib/meme-gate` and `worker/score/lib/nameability`, and it writes the number `0.4` out as a final verdict at four separate line numbers. So the product decision lives in the API-client folder, the thresholds live somewhere else again, and there is nowhere you could have looked that would have been wrong to look. Elsewhere in the same tree, a vendor file and a policy file import each other in a circle, and somebody has already hidden one of those imports inside a function to stop Node complaining. That is not sloppiness. It is the absence of a boundary.

Three layers, and only one of them thinks

The replacement has exactly three layers, and the whole design is one sentence: **logic never knows who it is talking to, and vendors never know what the logic decides.**

Adapters sit at the edge. Every outside thing — X, TikTok, Reddit, the Solana chain, pump.fun, Dexscreener, Jupiter, Rugcheck, Claude, the embedding model — gets exactly one folder, and that folder's only job is translation: take whatever that service hands back and restate it in our words. An adapter holds no thresholds, makes no product decisions, and contains no SQL. Adding a platform means writing one folder and one line in a registry, instead of forking the pipeline the way TikTok was forked into a second 340-line copy today.

The core holds all the logic, and is pure. No network, no clock, no database, no random numbers, no npm dependencies, and — this is the one that matters most — no platform names anywhere in it. Not *views*, not *retweets*, not *playCount*. Those are X's words and TikTok's words, and the moment they appear in the core, every future platform has to pretend it has them. That is precisely how TikTok's share count ended up in a column called *retweets* and how quotes — the reproduction signal the whole product rests on — became an honest-looking zero. Pure also means the core is the only part of the system you can replay: hand it yesterday's recorded inputs and it produces yesterday's answer, every time, with no key and no internet.

The record sits underneath. One database, one folder that is allowed to contain SQL, and every decision the core made written down as it happens. And one more wall past that: the app reads a separate, deliberately duplicated wire vocabulary that has no internal field in it, so it is not that the app *should not* show a score — it structurally cannot obtain one.

CONTRACTS

the shared words: Item · Observation · Decision
every threshold · the shape of every adapter

ADAPTERS – THE EDGE

One folder per outside thing. Its only job is to say what came back, in our words.

- X
- TikTok
- Reddit
- Replay
- pump.fun
- Raydium
- Dexscreener
- Jupiter
- Rugcheck
- Claude
- Gemini

no thresholds · no product decisions · no SQL · cannot import the core – the package manager refuses
adding a platform = one folder + one line in registry.ts

observations, already translated into our vocabulary

CORE – ALL THE LOGIC, PURE

The only place in the repository that makes a judgement. Seven verbs, seven folders.

- ADMIT
worth paying to watch?
- TRACK
when do we look again?
- DETECT
is it accelerating?
- GROUP
which story is this?
- QUALIFY
is it coinable?
- RESOLVE
which coin is real?
- RANK
what goes on top?

features/ one set of builders – the live system and the model trainer call the exact same function

no network · no clock · no database · no randomness · no npm dependencies · no platform words, ever

every decision, written down as it is made – inputs, verdict, reason, cost

STORE – THE RECORD

The only place SQL exists. One Postgres, three schemas, numbered forward-only migrations.

- items
- observations
- stories
- assets
- decisions
- labels
- carriers
- policies
- stage_runs

one translation module turns our words into the public ones

VIEW + APP – WHAT THE OUTSIDE WORLD SEES

A separate wire vocabulary with no internal field in it. The app imports that and nothing else.
the app cannot render a score because it has no way to obtain one

Reading down: the outside world comes in on the left of the top band, gets restated in our vocabulary, is judged once in the middle, is written down, and only then is translated back out. The rail down the left is the set of words all four layers agree on. Nothing skips a band.

The whole tree, and what every folder is for

Nine packages. There is no `utils/`, no `helpers/`, no `types.ts` — a junk drawer with a respectable name is still a junk drawer, and within a year nobody can safely delete anything from it. Two naming rules do almost all of the work. First, **every stage folder contains a file literally called `stage.ts`**, so "where is the admit logic" has a mechanical answer requiring no search and no memory. Second, **every number in the system lives in one file, `contracts/policy.ts`**, and a bare number typed anywhere in the core fails the build. That one is a direct response to the finding that made the last six months unauditale: every threshold in the current system is a number somebody typed, somewhere, and there is no list of them.

THE REPOSITORY, TOP TO BOTTOM		9 packages · ★ = the ones that carry the rule
insidor/	One repository. The name of a folder is the answer to a question.	
└─ README.md	Forty lines maximum, and it is a lookup table: question → directory.	
└─ contracts/	★ The words everyone must use. Definitions only — no working parts.	
└─ vocabulary.ts	The three nouns: an Item, an Observation of it, a Decision about it.	
└─ policy.ts	★ Every threshold in the system, in one frozen object. Nowhere else.	
└─ reasons.ts	The closed list of reasons anything was rejected — about sixty of them.	
└─ ports/	The shape an adapter has to fill: platform, venue, judge, store.	
└─ core/	★ ALL THE LOGIC. Nothing else in the repository makes a judgement.	
└─ admit/stage.ts	Is this post worth spending money to keep watching?	
└─ track/stage.ts	When do we look at it again — in ninety seconds, or in an hour?	
└─ detect/stage.ts	Is it speeding up compared with how it was behaving an hour ago?	
└─ group/stage.ts	Is this the same story as something we are already watching?	
└─ qualify/stage.ts	★ Is there a nameable, coinable thing here? ← the file in the test below.	
└─ resolve/stage.ts	Of the 306 coins named for this meme, which one is the real one?	
└─ rank/stage.ts	What goes on top of the board — and does it stay still while you read it?	
└─ kinetics/	Speed and acceleration maths, shared by every stage above.	
└─ features/	★ One set of feature builders. Live system and trainer call the same code.	
└─ decide.ts	The Decision object every stage has to return. This is how we learn.	
└─ adapters/	★ ALL vendor code. No thresholds, no SQL, and it cannot see the core.	
└─ platform/x/	X: search it, read it, restate what came back in our words.	
└─ platform/tiktok/	TikTok. Where plays became "views" and shares became "retweets".	
└─ platform/reddit/	Reddit. Same six files as the other two. That is the point.	
└─ platform/replay/	★ A "platform" backed by recorded files. Ships day one, costs nothing.	
└─ venue/solana/	The chain: pump.fun and the exchanges coins graduate onto.	
└─ market/	Dexscreener, Jupiter, Rugcheck — price, liquidity, and safety checks.	
└─ judge/anthropic/	Claude. The prompts live here as .txt files, never as inline strings.	
└─ meter/	One cost wrapper every paid call passes through. Nothing bills silently.	
└─ tape.ts	Record a vendor's answer once; replay it forever in tests.	
└─ store/	★ THE ONLY PLACE SQL EXISTS. One database, three schemas.	
└─ migrations/	Numbered and forward-only. 0001 through 0014. Never edited in place.	
└─ repo/	One small file per table. Nothing else in the repo writes rows.	
└─ projections/	★ The single module allowed to see both vocabularies, and translate.	
└─ view/	★ The public wire vocabulary. Duplicated on purpose — the duplication is the wall.	
└─ services/	The running processes. Thin wiring: adapters → core → store. No logic.	
└─ runner/	Six supervised loops in one process. About forty lines each. → Railway	
└─ chainwatch/	Watches new mints and owns a cursor that survives a restart. → Railway	
└─ watchdog/	Watches the other two. Deliberately on a different vendor. → Fly.io	
└─ app/	The website. Imports view/ and nothing else of ours. → Vercel	
└─ ml/train/	The only Python in the repository. Runs weekly, offline, never in the app.	
└─ eval/	Replays and blind tests. Forbidden from touching the network at all.	
└─ tools/	The six checks below. Plain scripts, no framework, run in about a second.	
└─ tapes/	Recorded vendor responses, committed to the repo. Roughly 40 MB.	
└─ legacy/	★ The current build, moved here whole and frozen. Bug fixes only. Still running.	

Nothing here is a category ("services", "components", "lib"). Every name is either a layer or a verb. The seven verbs in core/ are the seven questions the product asks, in the order it asks them.

The rule, and the fact that a machine checks it

A layered diagram is a wish unless something enforces it, and diagrams like this one have been drawn before and then quietly violated. So the rule is written as a list of who may import whom, and six separate mechanisms refuse a build that breaks it.

WHO IS ALLOWED TO IMPORT WHOM

read once — the shape is the whole design

contracts	→	nothing at all	
core	→	contracts	never a vendor, never a database
adapters	→	contracts	never core
store	→	contracts view	never core, never adapters
view	→	nothing at all	
services	→	everything — this is the only place things get wired together	
app	→	view	never core / adapters / store
eval	→	contracts core store view	never adapters
watchdog	→	contracts store	never another service

app must never import core

The product rule is that our reasoning is never on screen. **Six surfaces leaked scoring in the last build**, and three of them were reasonable-looking UI written against data the browser happened to already have. If the app cannot reach the scoring code, it cannot render a score.

eval must never import adapters

A replay that can reach the internet can look up what happened afterwards. **That is exactly how the last validation harness marked its own homework**. Removing the ability is cheaper and more reliable than remembering not to.

watchdog must stand alone

The process that tells you the system died must not share code with the system. **It runs on a different vendor** for the same reason: an observer that shares a failure with the thing it observes tells you nothing at the moment you need it.

Nine lines. If a new person reads only one thing about this codebase, it should be this table — it says both what the pieces are and what they are forbidden to know.

HOW IT IS ENFORCED — FOUR LAYERS, CHEAPEST FIRST

all six checks run in under a second

LAYER 1

the package manager

The adapters package simply does not list the core as a dependency, so the import fails on the developer's own laptop. **This was measured:** under npm it silently worked anyway. Under pnpm it dies. That is why the stack says pnpm.

LAYER 2

the type checker

One line of config stops someone routing around the rule with a relative path like `../core`. The compiler you already run reports it as an error, with a file and a line number.

LAYER 3

the import map

A tool walks every import in the repository against an allow-list — not a list of forbidden pairs, which always misses one, but a list of what each package **may** reach. It also fails on circles, the exact defect in the build today.

LAYER 4

the word check

A leak usually arrives as a **field name**, not an import. So a script fails the build on any platform, chain or vendor word inside core or contracts — **views, retweet, playCount, pump.fun, solana, claude**.

```
$ pnpm check
$ pnpm test
$ pnpm graph && git diff --exit-code docs/graph.svg
```

The last line draws the picture at the top of this section from the actual code and fails if it changed. **A changed diagram in a pull request is a signal a non-engineer can read.**

The greps stop accidents, not adversaries — which is the correct threat model for a two-person team. The real guarantees are the three layers behind them.

The test: how many seconds to the right file

Here is the actual measure of whether any of this worked. A developer — or you, or a person hired next year — wants to change how a post is judged worth making a coin out of. No README, no search, no asking anybody.

0s It is a judgement. Judgements live in one place and that place is called <code>core/</code>. Adapters cannot make one; the store cannot make one; the app cannot see one.	3s The folders are the verbs. admit · track · detect · group · <code>qualify</code> · resolve · rank. "Is there a coinable thing here" is qualify. There is nowhere else it could be.	7s Every stage has a <code>stage.ts</code>. Not <code>index.ts</code>, not <code>qualifier.ts</code>, not <code>helpers.ts</code>. The rule holds in all seven folders, so it needs no memory.	11s Open it. <code>core/qualify/stage.ts</code> — about 120 lines, no network calls, no numbers in it. The numbers it uses are named, and they are all in <code>contracts/policy.ts</code>.
---	--	---	--

Eleven seconds, three decisions, zero searches. The same walk finds every other judgement in the system, because the tree has exactly one shape.

And two things follow that are worth more than the eleven seconds. Because the file has no numbers in it, changing a threshold is a one-line edit in a file that lists every threshold in the product side by side — you can see what else you are near. Because the file has no network calls in it, you can run the change against every decision the system has already made and see what would have changed, before shipping it. Neither is possible today.

THE SAME QUESTION, IN THE BUILD WE HAVE <pre>worker/adapters/anthropic/meme-score.js ← a vendor folder ├ requires worker/score/lib/meme-gate ├ requires worker/score/lib/nameability └ writes 0.4 as a final verdict at 4 line numbers</pre> The product decision is inside the folder named after the model provider. The thresholds it applies live in a third place. Nothing in the folder names points here. Meanwhile a vendor file and a policy file import each other in a circle, and one of those imports has been hidden inside a function to keep Node from noticing.	THE SAME QUESTION, AFTER <pre>core/qualify/stage.ts ├ imports contracts ← types and thresholds only ├ no fetch · no clock · no database · no vendor └ every number named, defined in contracts/policy.ts</pre> The model's opinion arrives as an ordinary field, handed in by the service that called it. If it is missing, the stage abstains with a named reason rather than guessing. Which means the same file can be replayed over months of past decisions with no key and no internet, and give the same answer every time.
--	---

This is the difference the whole section is about. Not tidiness — findability, and the ability to change one number and know what it does.

WHY THIS IS WORTH THE DISCIPLINE The tree is the documentation, and it is the only documentation that cannot go stale. A README drifts from the code within weeks. Folder names cannot: the build fails if the code stops matching them. Adapters that import the core do not compile. Numbers typed into the core do not compile. Platform words inside the core do not compile. A person joining in a year reads the tree, not a document, and the tree is true by construction. None of this is extra work bolted onto the schedule. It is the same code, in different folders, with six one-second checks in front of it — and it is the difference between "add Reddit" being a folder and a registry line, versus what adding TikTok actually cost: a second pipeline, a second set of parsing bugs, and a core signal quietly reading zero.
--

13. How each part gets built

Eleven steps. Each one ships on its own and is useful on its own, and each one ends with a test that either passes or does not — never with somebody deciding it feels finished.

A build plan written as a list of things to make is a wish. The same plan written as a list of things to *prove* is a schedule. So every step below carries a **done when** line, and every one of those lines is something a machine can check: a query that returns the right shape, a test that goes red when a threshold moves, a folder whose diff is empty. Nobody has to judge whether the storyteller is good enough. Either the gold set produces a curve or it does not.

The order is not a preference. It comes from one fact: three of the five data streams this business depends on are **not being recorded at all today**, and a fourth deletes itself every twelve hours. Code can be written later. A week of mint times that were never written down is gone forever. So the steps that start a recorder come before the steps that build anything to look at, even though the recorders produce nothing a person can see.

Three things to notice before the list

STEP 1 • COUNTER-INTUITIVE

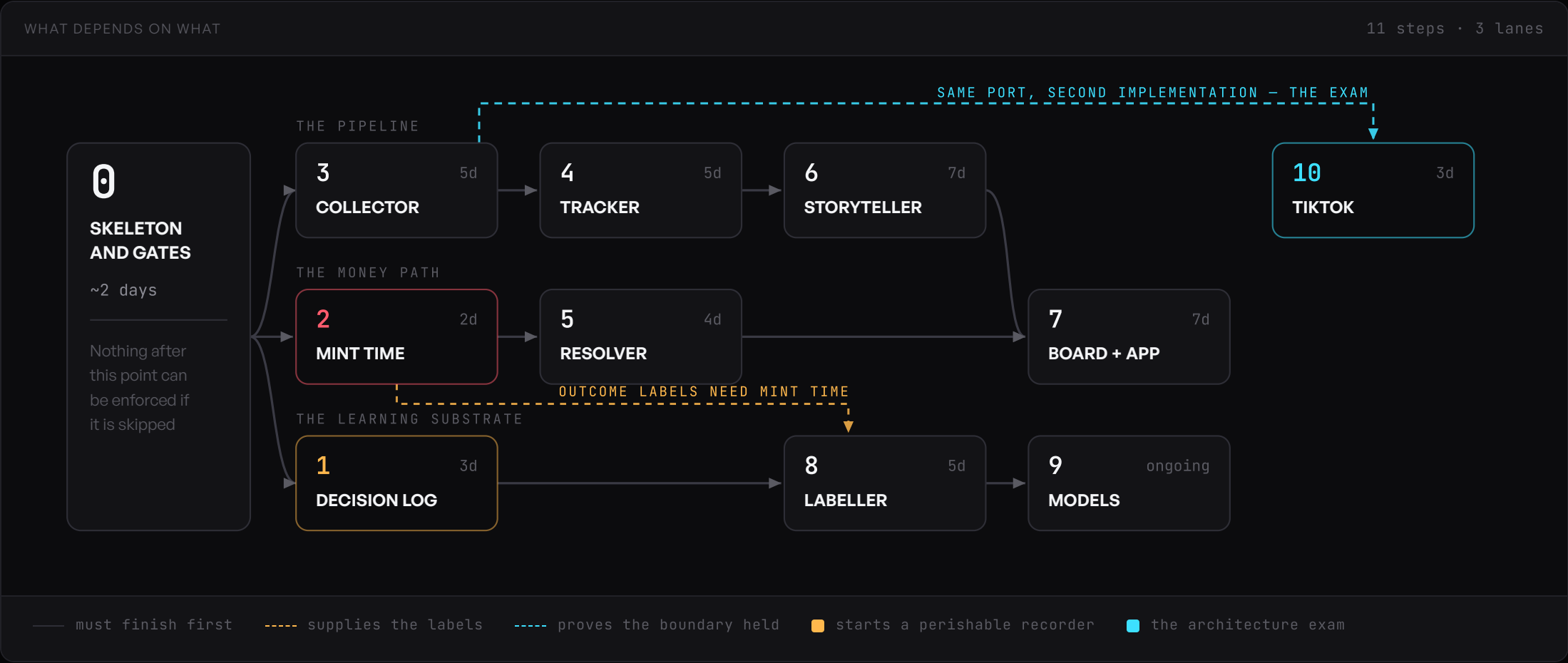
It goes into the old build
The decision log is retrofitted into the pipeline running **right now**, not the new one. That pipeline makes a few thousand judgements a day and writes down **none** of them. Every day spent building the replacement first is another day of training data thrown away.

STEP 2 • CANNOT MOVE

Mint time is a deadline
Ten of the eleven steps can be reordered under pressure. This one cannot. Without a stored mint time there is no way to say afterwards whether a call was early or late — so every day it is not running is **label data that will never exist**, at any price.

STEP 10 • THE EXAM

TikTok is the test, not a feature
Adding the second platform should be one new folder and one new line. If it has to touch the thinking layer, the whole design failed — and it failed **cheaply, three days in**, instead of a year later when someone says a new platform is a rewrite.



Read the diagram as three lanes that run mostly in parallel. The top lane is the machine that finds things — collector, tracker, storyteller, and the screen people look at. The middle lane is the money path — knowing when a coin appeared, then choosing the right one, then putting a Buy button next to it. The bottom lane is the part that lets the system get better over time — write down every judgement, join it to what actually happened, then let a model learn the join. The lanes touch in only two places, and both are drawn dashed, because both are the kind of dependency people forget and then discover late.

The eleven steps

STEP 0 ~2 days	The skeleton and the gates Nine folders with rules about which may talk to which, one file holding every threshold in the system (today there are sixty of them typed into twenty-four different files), six automated checks, and the current build moved wholesale into a folder called Legacy/ where it keeps running untouched.
----------------------	---

FOUNDATION

UNBLOCKS

Everything. Boundaries added after the code exists never actually get added — they get argued about and waived.

DONE WHEN

A change that lets vendor code reach into the decision code fails the build and names the rule it broke.

STEP 1
~3 days

PERISHABLE

The decision log — into the build that is running today

HIGHLIGHT 1

Three tables and one small writer, wired into the **existing** cron runner and the existing gate functions. Nothing is rewritten. Every judgement gets stored with what it saw, what it chose, and which thresholds it was judged against. The 2% holdout and the 5% exploration reservation land in the same commit, because a log of decisions you always would have made teaches nothing.

UNBLOCKS

Replay, the first training set, and the first model roughly five months out. It also fixes a live bug: a stage that dies mid-run currently looks identical to a stage that ran and found nothing.

DONE WHEN

One query separates ran and found nothing from died, and a process killed halfway through leaves its row with no finish time rather than no row at all.

STEP 2
~2 days

CANNOT MOVE

Mint time, stored, with a source attached

HIGHLIGHT 2

A table of coins, filled by polling the launchpad's own list and confirming each one against a single generic chain call — plus a **coverage log**, so a stretch where the poller was down is recorded as *we were not watching* rather than quietly counted as *no coin existed*. That second half is the whole point: the failure to avoid is a gap that flatters us.

UNBLOCKS

Every outcome label, the rule that stops us claiming credit for something posted after the coin, and every how-far-into-the-run number the product will ever publish — a coin's age against a six-day arc is the whole reason a user can tell a run that is still climbing from one that is over.

DONE WHEN

A gap induced on purpose produces a coverage row, and the labeller marks that window unmeasurable instead of scoring it as a miss.

STEP 3
~5 days

PIPELINE

The collector

The X connector and the first decision stage — *is this worth spending money to watch?* — running alongside the old ingest without replacing it. Critically, a **second** connector ships in the same step: a fake platform backed by recorded files. An interface with one implementation is always secretly shaped like that implementation.

UNBLOCKS

Step 10, which is only cheap because a second implementation already exists. It also gives the testing tools a data source that needs no network and no vendor bill.

DONE WHEN

One test suite passes unchanged against both the real platform and the recorded one, and no platform's name appears anywhere in the thinking layer.

STEP 4
~5 days

PERISHABLE

The tracker and the honest counters

When to look at a post again, how to store each reading, and the rule that says a rounded counter which has not visibly moved means **we learned nothing** — not that the post is cooling. Today the system publishes a zero there, and zero reads downstream as fading, which demotes exactly the posts that are picking up speed.

UNBLOCKS

Every speed-of-growth signal, and the curve the board is built on. Perishable: a day not shipped is a day of curve data that will never exist.

DONE WHEN

A rounded counter whose change is smaller than its rounding step records censored: below step — and no rate at all.

STEP 5
~4 days

MONEY PATH

The resolver — which coin, if any

Eight checks in cheapest-first order, candidates generated by time rather than by name, a margin rule so two plausible coins produce *no answer* instead of a coin flip, and a hard split between a ticker we actually saw and a ticker we guessed. **This is the only step where being wrong costs a user money** — 189 of 323 currently resolved tickers share no words at all with the story they are attached to, and a working swap sits one click away.

UNBLOCKS

Turning the Buy button back on without it being wrong. Nothing else in the plan is allowed to ship a Buy affordance before this.

DONE WHEN

A frozen set of the real past mistakes runs on every change, so a threshold cannot be nudged silently.

STEP 6
~7 days

The storyteller

Deciding that forty posts are one story rather than forty stories. The free, deterministic signals first and alone — the same image, the same phrasing, the same sound or effect — measured before any meaning-matching is switched on. The English-only filter comes off in the first hour of this step; it currently

PIPELINE

appears in two places, so removing one leaves half the funnel shut while looking fixed.

UNBLOCKS

Grouping across platforms, which is the product's actual premise. Today the grouper leaves 91% of stories with exactly one post in them.

DONE WHEN

A hand-labelled set of 300 pairs produces a precision/recall curve, and the joining threshold is read off that curve rather than typed in.

STEP

7

~7 days

PRODUCT

The board and the app on the new store

The screen people actually use, reading from a public view of the data that contains only facts about the world. The ordering machine — ranks committed with a tick number so the list cannot shuffle under the cursor — moves across verbatim, since it already works.

UNBLOCKS

Everything user-facing, and it is the step that makes the show-facts-not-reasoning rule structural rather than a habit.

DONE WHEN

Searching every screen file for our internal scores and gate names returns nothing, and the app is forbidden by rule from importing the thinking layer at all.

STEP

8

~5 days

LEARNING

The labeller and replay

Joining what we decided to what actually happened — every label carrying the population it was drawn from, so winners and losers are never quietly measured against different denominators. Then the machine that re-runs old decisions under new rules. The existing outcome labeller carries forward: it is the best artefact in the current repository, separating winners at 18× their median peak from losers at 1.19×.

UNBLOCKS

Every model in step 9, and the ability to change a threshold and see who it would have affected before shipping it.

DONE WHEN

Changing one number in the thresholds file produces a list of last month's decisions that would have flipped — with no deploy and no network access.

STEP

9

ongoing

CALENDAR-BOUND

The models

Learned judgement replacing typed numbers, one stage at a time, in the order the labels arrive: the coinable-or-not call first because its label is free and plentiful, then what to watch, then what is accelerating. **Ranking last**, because it needs a record of what people were shown, and that record does not exist yet. This step is bound by the calendar, not by effort — about 2,000 labelled stories are needed before the first model is worth training.

UNBLOCKS

The compounding advantage. Nothing else in the plan improves on its own once shipped.

DONE WHEN

A challenger runs silently alongside the incumbent for seven days and is promoted by a database row, not by a deploy.

STEP

10

~3 days

THE EXAM

The second platform

HIGHLIGHT 3

One folder for TikTok and one line in a lookup table. The step exists to answer a question about the architecture rather than to add a feature. And it has a specific thing to get right: TikTok has **no reproduction count at all**, so the connector must declare that concept *absent* rather than write a zero — which is precisely the bug live in the build today, where every TikTok post reports its most important signal as an honest-looking nought.

UNBLOCKS

Nothing downstream. What it buys is the answer to whether the previous ten steps were worth doing.

DONE WHEN

The change is one new folder plus one line — and the diff against the thinking layer is empty.

WHAT THE NUMBERS ADD UP TO				honest, not optimistic
43	67	17–18	3	
Developer-days if you only count writing the code, which is what the step estimates above measure	Developer-days once backfills, cutover switches, comparison queries and keeping the old system alive are counted	Weeks at a realistic pace for one developer who is also nursing a live system. About four months	The week after which a slip costs time rather than the asset — every perishable stream recording	

THE ONE PLACE THE ORDER CAN BEND

Steps 1 and 5 are the two that repair live defects. If the Buy button matters more right now than the board does, **step 5 can jump ahead of steps 3 and 4** with no structural cost — the resolver needs stored mint times, and nothing else from the pipeline lane.

Everything else can slide too. **Step 2 cannot**, and it is worth saying why one more time: the other ten steps trade weeks for weeks. This one trades weeks for data that cannot be bought, borrowed or reconstructed afterwards. The same logic puts step 1 in the old build rather than the new one — a few thousand judgements a day are being made and thrown away while this deck is being read.

And the step most likely to be cut under deadline is step 10, because by then the product works and a second platform feels like a nice-to-have. Cutting it means **never finding out whether the architecture was real** — three days from the end of a four-month build.

14. The timeline, honestly

This is about four months of work, not two. An earlier version of this plan said eight to nine weeks; that number counted the code and nothing else, and it was off by roughly double. The honest figure is 17 to 18 weeks. What makes four months tolerable is not the total — it is the shape. One date in it matters more than the finish line.

The earlier estimate came from adding up the build steps: write this stage, write that adapter, wire the store. Forty-three developer-days. That arithmetic is roughly right for the code sitting on its own in a fresh folder. It is roughly half the calendar, because it quietly assumes three things that are not true here. It assumes nothing has to be moved from the old system to the new one. It assumes the two never have to run side by side while we check they agree. And it assumes the developer is not simultaneously keeping a live system alive — a system whose record includes a seven-hour silent outage and a heartbeat that stayed dead for two weeks.

Put the missing work back in and the figure is about sixty-seven developer-days. Then divide by a real week rather than an ideal one. One developer, a production system that occasionally demands attention, and a non-engineer reviewer does not produce five clean days a week; three and three-quarters is the honest rate. Sixty-seven divided by 3.75 is 17.9 weeks.

Where the missing twenty-four days were

the earlier estimate priced the first row only, and priced it optimistically

WORK ITEM	DEVELOPER-DAYS
Coding, re-estimated with the transition included	50.5
Backfill migrations, and verifying they worked	3.0
Cutover switches, shadow plumbing, rollback	3.0
Comparing old against new while both run, plus one alarm	3.0
Keeping the old system alive — half a day a week × 16 weeks	8.0
Total	67

EFFORT

67 developer-days

DIVIDED BY A REAL WEEK

÷ 3.75 productive days / week

CALENDAR

17.9 weeks ≈ 4 months

THE EARLIER NUMBER

~~8-9 weeks~~ counted the code in isolation. It was wrong by about **2×**. Nobody was being dishonest — the transition work simply had no line on the page.

Not in any of these numbers: **the models**. Those are calendar-bound rather than effort-bound — they need thousands of recorded decisions to exist before anyone can train on them. Month five at the earliest, and only if the decision log ships in week two.

The shape is the good news

Four months is only frightening if you believe the whole thing is worthless until the last day. It is not. The value arrives in a specific order, and the most valuable week is the third one, not the eighteenth.

Five streams of data in this business are perishable. They describe a moment — what a post's view count was at 3:12pm, what we decided about it, what the coin was worth ninety seconds later, how common that phrase had been over the previous fortnight. If you do not write them down as they happen, they are gone. You cannot go back and get them, at any price, ever. Three of those five are not being recorded at all today. A fourth is being recorded and then deleted twelve hours later. That is the asset the company is actually accumulating, and right now most of it is running down a drain.

The five perishable streams — status today

every day before week 3 is a day of this permanently lost

01

The decision log

Every judgement we make, the reason, and what happened afterwards. A few thousand a day.

NOT RECORDING

02

Mint and price series

What each coin was worth, minute by minute, from the moment it existed.

NOT RECORDING

03

Phrase frequency, 14-day rolling

How common each phrase was *at the time* — which is what makes a phrase new.

NOT RECORDING

04

The near-miss archive

The posts we looked at and turned down. The only source of negative examples we will ever have.

DELETES ITSELF EVERY 12H

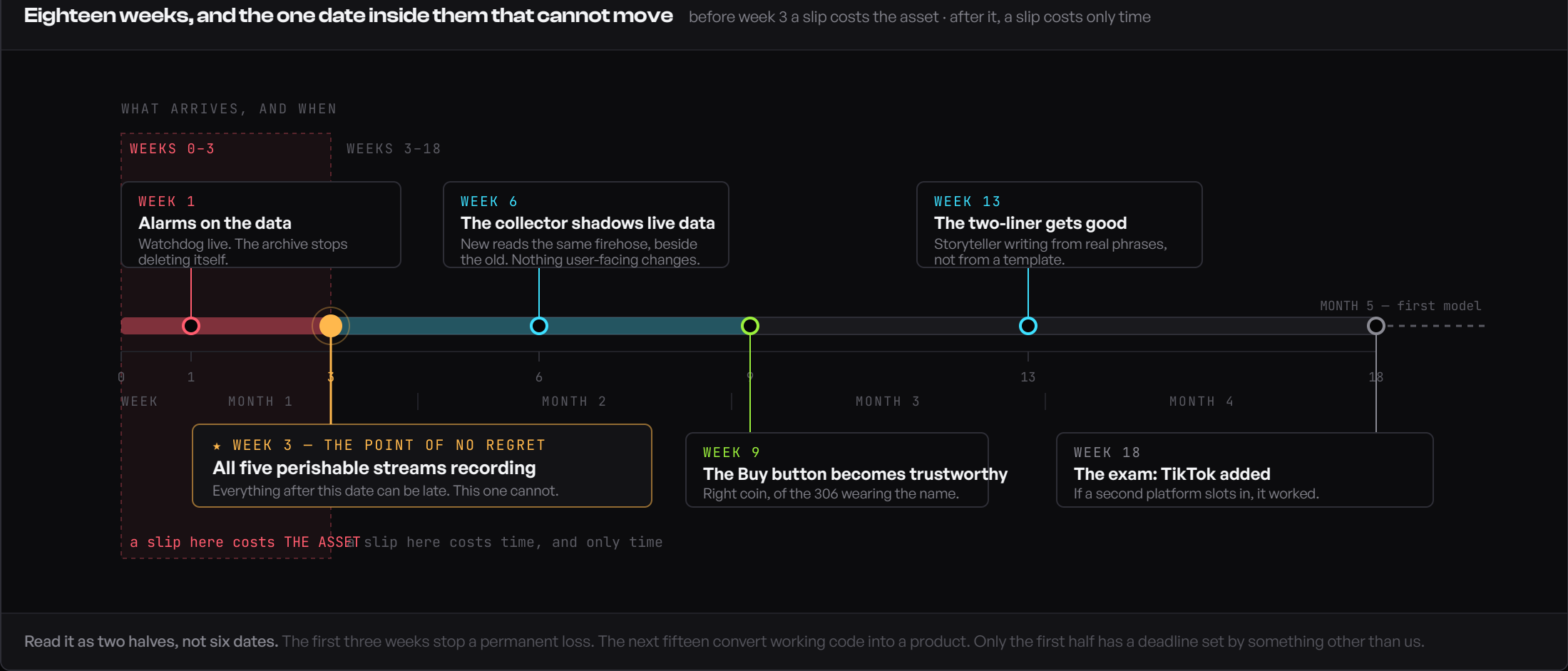
05

View-count snapshots

Recording — but how long a post is watched depends on how well it did, so the record teaches its own answer.

RECORDING, UNUSABLE

Why this dates the plan. Code can be written later. These cannot be collected later. Week 3 is the week all five become honest recordings — and it is the reason the order of the build is not negotiable even though the finish date is.



PART FOUR

How it comes together

The line between what we compute and what we show, and one story followed all the way through.

-
- 15 Where the line sits
 - 16 One story, end to end

15. Where the line sits

Part four is where the two halves of this document meet. Everything before this was the machine; everything after is the screen. Exactly one rule governs what is allowed to cross between them, and it has already caused one version of this deck to be thrown away.

The rule is short. **Facts about the world go on screen. Our reasoning never does.** A view count is a fact — the post really has 2.4 million views whether or not Insidor was ever built. A market cap is a fact. So is the minute a post went up, the minute a coin was created, the name of the account that posted it, and how many wallets hold the token. All of those existed before we looked and would carry on existing if we shut the servers down tomorrow.

Our reasoning is a different kind of thing. When the system decides a story is worth showing, it is doing arithmetic: comparing a view count against the same post’s view count twenty minutes earlier, weighing how many separate accounts are telling the same ferry-captain story, asking Claude how memeable the thing is and getting a number back. Those numbers are real, and they decide the ordering of the entire feed. But they exist only because we invented them. They are our workings, and workings do not go in front of a user.

In the founder's words: *"The app's job is to make the judgement so the user doesn't have to. Showing our workings would just make them do our job for us."* That is the whole argument. A trader opening Insidor at 09:13 has perhaps ninety seconds. Hand them a heat score of 8.4 and an acceleration ratio of 3.1x and we have not helped — we have added a second decision on top of the first one and asked them to learn our private scale before they can use the product. The correct output of all that arithmetic is not a number. It is an ordered list and one plain sentence.

ON SCREEN	NEVER ON SCREEN
Facts about the world	Our reasoning
True regardless of Insidor. Anyone could verify them from the original post, a block explorer, or a price feed.	Exists only because we computed it. Meaningless outside our own system, and impossible for a user to check.
How many people have seen the post2.4M views	How memeable Claude scored it meme_score 7
How old the post is4h · posted 09:00	Growth rate between two checks+18.4% / 20m
Who posted it, and where@nordkyst_ferge · X	Acceleration ratio3.1x
Whether views are still climbinga rising line	Combined ranking scoreheat 8.4
When the coin was createdminted 22m ago	Eligibility gates and thresholds author_cap 2
Market cap of the coin\$799.6K	How sure we are a coin matches match 87%
Liquidity in the pool\$84.2K	How many same-name tokens we discarded305 of 306 dropped
How many wallets hold it1,204 holders	Position in the ranking, and why rank ▲4
Price change over the last hour+38.4% · 1h	Which model made the judgement model_name
All of these can be checked by the user somewhere else. Showing them builds trust rather than spending it.	Every one of these is stored, logged and used. None of it is rendered. The user gets the decision, not the derivation.

Left column: nine things a user can independently verify. Right column: nine things that only mean something inside our own database. The struck-through values on the right are printed here, in this document, so the team can recognise them on sight — they never appear in the product.

The same row, twice

Here is the identical story — Harbor Last Horn, post four hours old, three coins already minted from it — rendered both ways. The left is what an earlier draft put in front of a user, and why that draft was rejected. The right is the product.

One thing changed since that draft, and it changed the words rather than the layout. We measured the gap between a post going up and a coin being minted from it: the median is 3.8 minutes, and for the coins that go on to clear ten million it is 1.1 minutes. Bots get there first, always. So the promise can never be "we show you the story before the coin exists." What is still true, and still worth paying for, is that the run takes about six days — the peak comes a median of six days after the post, and no coin above a million ever peaked inside the first hour. Being early to the mint was never the product. The honest claim is therefore: **several coins were minted from this story, this is the one that is really about it, and it is still**

climbing. Every clause of that is a fact. The row carries the first and last of them; the middle one is the story page's whole job, and the row's single button is the door to it.

NOT THIS

the machine talking to itself, in public



Harbor Last Horn▲4

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

@nordkyst_ferge · 2.4M · 4h · match 87%

HEAT8.4

ACCEL3.1x

ORG3

MEME7

Δ20M+18.4%

40 of 63 eligible · 12 hidden by gate · 305 same-name tokens dropped · author cap 2

THIS

the judgement, already made



Harbor Last Horn

A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.

2.4M views · 4h old

Still climbing

3 coins →

Same story, same underlying numbers, same ranking. On the left the user is asked to interpret five scales nobody taught them. On the right they get the seven things every feed row carries and nothing else: a picture, a title, two lines of plain English, the view count, a line still going up, how old the post is, and one button. The button says 3 coins rather than Buy \$FERRY because three were minted from this story and choosing between them is a second question with its own screen — section 03, where there is room to show the evidence. The reasoning behind the ordering is identical in both rows; only one of them says so out loud.

The harder cases

Obvious internals are easy to keep off screen. The rule earns its keep in the awkward cases, where a number would be genuinely convenient and still has to stay behind the curtain — and in one case where a number that looks like a fact turns out not to be one.

A Coins get labels, not confidences

\$FERRY

FIRST

minted 09:31

\$799.6K

\$HORN

BIGGEST

minted 10:14

\$1.2M

\$BOAT

UNVERIFIED

minted 11:02

—

INSTEAD OF match 94% → 88% → 31%

B Climbing is a shape, not a percentage

ATTENTION
Still climbing
2.4M views · 4h

INSTEAD OF +18.4% / 20m → accel 3.1x

C Unsure means no button at all

\$BOAT

UNVERIFIED

We can't confirm this coin is about this story. [Open it on DexScreener](#) to check for yourself.

INSTEAD OF Buy — disabled

D The sentence is the output

“A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back.”

@nordkyst_ferge on X · 2.4M views · 4h old

BEHIND IT three collectors, twelve re-measurements, one Claude judgement, one grouping decision

E A fact has to be the thing it says it is

X

@nordkyst_ferge2.4M views

TIKTOK

@harbourclips890K plays

INSTEAD OF 890K views → a play count wearing another name

F The order is the opinion — never numbered

Harbor Last Horn2.4M · 4h3 coins

Paid to Move to Norway1.1M · 1hno coin yet

Plastic Plant Grandma850K · 2h\$PLANT \$562.8K

INSTEAD OF #1 → #2 → #3 → showing 40 of 63

A. FIRST, BIGGEST and UNVERIFIED are things a user can act on; 94% is a thing they have to interpret — and since only about one coin in twenty declares the post it came from, that 94% is our inference dressed as measurement. B. A rising line says "still climbing" faster than any percentage and cannot be misread as a promise. C. A disabled button is a taunt — it says we know something and won't tell you; nothing, plus a link out, is honest. D. The two-liner is not a summary of the reasoning, it is the reasoning, delivered. E. Our TikTok parser writes play counts into a column called views; a mislabelled fact is worse on screen than an omitted one. F. The order is our loudest

opinion, so it is expressed as order and nothing else — no positions, no totals, no count of what we filtered out.

The test

Any new screen, any new field, any argument about whether something belongs — this settles it in about five seconds, and anyone on the team can apply it without asking permission.

BEFORE YOU PUT A NUMBER ON SCREEN, ASK

Would this number still be true if Insider did not exist?

YES → IT CAN BE SHOWN

2.4M views — the post has them either way

\$799.6K — on-chain, public, checkable

minted 22m ago — the block says so

1,204 holders — anyone can count them

NO → IT STAYS BEHIND THE SCENES

meme_score 7 — we invented the scale

accel 3.1x — needs our own history to exist

match 87% — our uncertainty, not theirs

rank ▲4 — a fact about our list, not the world

Second question, for anything that passes: is it still the quantity its label claims? A TikTok play count printed under the word "views" passes the first test and fails this one.

One question, applied to every field, plus a second one for anything that survives it. Useful side effect: the numbers that pass are exactly the numbers a sceptical user can verify somewhere else, so showing them buys credibility. The numbers that fail are the ones we could never prove anyway.

There is a discipline hidden in this that is worth naming. A product that never shows its scores cannot hide behind them. If the ordering is wrong, the user sees a bad story at the top and nothing else — no 8.4 to argue with, no gate to blame. The line does not only protect the user from our arithmetic. It removes every excuse for the arithmetic being poor.

16. One story, end to end

One morning, minute by minute, with the machine from Part Three running underneath it. The version of this section in the last deck had the user minting the first coin. The timing measurements killed that. Here is the same morning told honestly: the coins already existed before we had even seen the post, and it made no difference at all to what the user made.

At 09:00 a Norwegian account called [@nordkyst_ferge](#) posts forty seconds of video. A retiring ferry captain blows his horn one last time, and every other boat in the harbour blows theirs back. It is a lovely thing and nobody in crypto is looking at it.

Sixty-eight seconds later somebody's bot mints **\$FERRY**. At 09:03 a different bot mints **\$HORN**. By 09:06 there are a dozen coins off that one clip. None of them was made by a person who watched the video and thought it was funny; they were made by software that saw a view count climbing and fired. Our collector does not see the post until **09:04** — by which point two coins are already trading.

That is the part that used to be the pitch and is now just background weather. The interesting question is what happens next, because the coin that eventually matters does not peak for another six days.



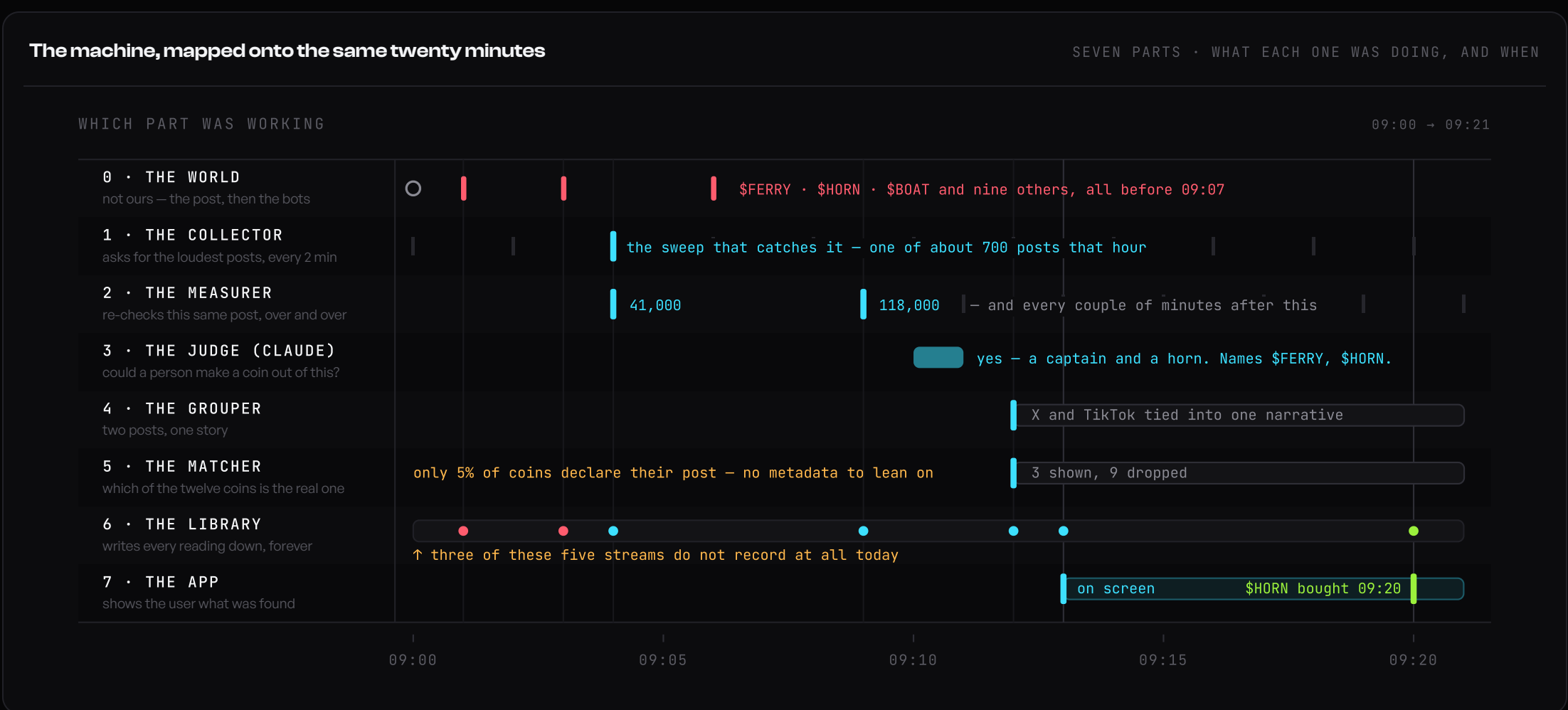
Which part of the machine was doing what

At **09:04** the collector did what it does every two minutes: asked our data reseller for posts pulling unusual attention right now. The ferry clip came back at 41,000 views. On its own that number is worthless. It could be a post that exploded in four minutes or one that crawled there over two days, and nothing in the reply tells you which.

So the measurer put it on a list and asked again. At **09:09** the same post read 118,000. That second reading is the whole trick, and it is the one thing on this page nobody will sell us later. A view count is a snapshot anyone can buy; two view counts on the same post at two known times is a fact about the world that only exists if we were standing there writing it down.

The judge — Claude, reading the post, not a model we trained — answered one question at **09:10**: could a person plausibly make a coin out of this? Most fast-climbing posts fail that. Politics, sport, a plane crash. A retiring captain and a harbour full of horns passes easily and arrives with its own names attached. At **09:12** the TikTok clip surfaced and the grouper tied the two into one story.

Then the part that is genuinely new. Twelve coins had been minted off this clip, and this is the problem the deck used to skip: a pump.fun search for "chill guy" returns **306 tokens**, and only about 5% of coins over \$50K declare which post they came from. So the matcher does not read metadata, because there is none. It works from mint time against post time, ticker against the words in the clip, and the shape of the money. It came back with three worth showing and nine not worth the row.



What today's build would have kept from that morning

Almost none of it, and this is the reason week three of the plan is the point of no regret. The diagram above describes seven parts working. The code we have today runs most of them and *remembers* almost nothing, which means a morning like this passes through the system and leaves no trace we could ever learn from.

THE MOMENT	WHAT THE DIAGRAM SHOWS	WHAT TODAY’S BUILD ACTUALLY KEEPS
09:04 · 09:09 THE TWO READINGS	41,000 views, then 118,000 five minutes later. The comparison that says “still climbing”.	Not reliably. The speed figure has no as-of filter, so asking for it tomorrow returns a different quantity under the same name. Snapshots stop three hours in, and slow posts are dropped after ten minutes — so the length of a post’s history is decided by its outcome.
09:10 THE JUDGE’S ANSWER	Yes, a person could make a coin out of this — and here is the sentence to put on the card.	Nothing. A few thousand of these decisions are made a day and not one is written down. We cannot tell you what we said yes to last Tuesday.
09:12 THE TIKTOK CLIP	890,000 plays, a second independent source, the thing that turns one lucky post into a story.	Wrong, quietly. Plays are written into a column called views and shares into one called retweets. Quotes — the reproduction signal the whole product rests on — is hard-coded to zero, because TikTok has no equivalent. So every TikTok clip reports an honest, meaningless zero.
09:20 · DAY 6 WHAT HAPPENED NEXT	\$HORN was the right pick out of twelve, and it ran for six days.	Nothing at all. No record links the story to the coins, so there is no way to ask later whether we were right. This is the row that makes the other three worth fixing.

Everything in the right-hand column is a week-three problem, not a month-four one. Data you did not record is not late — it is gone.

What the user actually did

Very little, which is the point. They opened Insidor while waiting for a coffee. Near the top of the feed was one sentence — “*A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back*” — a still from the clip, the view count, how old it was, and a small line still bending upwards. They tapped through, watched forty seconds of video, and decided the horn was funny enough that other people would find it funny too.

Then came the part no other screen can do for them: three coins, in a list, each labelled with a fact rather than an opinion. Which was minted first. Which is biggest right now. Which has no traceable link back to either post. They picked **\$HORN** and bought it at 09:20, seven minutes after the story appeared and seventeen after the coin existed.

STORY · HARBOR LAST HORN					09:20
 A retiring ferry captain blew his horn one last time, and every boat in the harbour blew back. @nordkyst_ferge on X · 2.4M views · 20 min old also a TikTok clip by @harbourclips · 890K plays · 8 min old					
COIN	MINTED		MARKET CAP	LIQUIDITY	HOLDERS
\$FERRY FIRST	09:01		\$41.2K	\$9.8K	214
\$HORN BIGGEST	09:03		\$86.9K	\$18.4K	631
\$BOAT UNVERIFIED	09:06		\$2.1K	\$1.3K	27
\$FERRY, six days later peaked \$799.6K		\$HORN, six days later peaked \$1.2M		\$BOAT, six days later peaked \$11K	

The screen at 09:20, and what those three coins went on to do. Every number here is a fact about the world — mint times, market caps, liquidity, holders. None of them would change if Insidor did not exist. The bottom strip is hindsight, shown here for the deck; the user saw only the rows above it.

ON THE SCREEN

- The sentence: **a retiring ferry captain blew his horn one last time, and every boat in the harbour blew back**
- Both clips, and who posted them — **@nordkyst_ferge** on X, **@harbourclips** on TikTok
- 2.4M views · 890K plays · 20 minutes old**
- Three coins with their **mint times, market caps, liquidity and holders**
- Which was **first**, which is **biggest**, and which has **no traceable link** back to either post

NEVER ON THE SCREEN

- That we compared 41,000 at 09:04 with 118,000 at 09:09, or what that comparison told us
- Why this story sat above four others in the feed
- What the judge weighed before saying yes
- How the matcher dropped nine coins and kept three
- Every threshold, cut-off and cap in between

Left column: facts about the world. Right column: our reasoning. The user gets the decision; working it out is our job, not theirs.

One morning is not evidence

This is a reconstruction of a good day, and it should be read as an illustration of the shape rather than a claim about the hit rate. The honest version of the pitch fits in one line and no longer contains the word *first*: **of the things minted in the last hour, this is the real story, and it is still climbing**. That claim survives the timing data. The old one did not.

What we do not yet know is how often a morning like this actually happens — how many of last week’s winners we would have seen at 09:04 rather than 09:47, and how often the coin we labelled biggest was the one that ran. Both questions are answerable, and the only thing standing between us and the answer is that nobody has been writing the mornings down.